Design and Implementation of a Force Controller for a Haptic Device

PANTELIS MPONITSIS

Supervisor Professor: Kostas Vlachos

Diploma Thesis submitted in partial fulfilment of the
requirements for the Diploma in
Computer Science and Engineering

October 11, 2025

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

UNIVERSITY OF IOANNINA

Abstract

Η εργασία εξετάζει τον έλεγχο δύναμης σε απτικό μηχανισμό πέντε συνδέσμων με δύο ενεργές και τρεις παθητικές αρθρώσεις. Αρχικά, υλοποιούνται η ευθεία και η αντίστροφη κινηματική ανάλυση, η διαφορική κινηματική ανάλυση, με εύρεση του Ιακωβιανού πίνακα, και η δυναμική μοντελοποίηση. Έπειτα εφαρμόζεται έλεγχος θέσης PID για κατανόηση της επίδρασης των τριών κερδών. Στην συνέχεια, ακολουθεί η μοντελοποίηση της ανθρώπινης επαφής, μέσω ενός ζεύγους ελατηρίου - αποσβεστήρα και υλοποιείται ελεγχος δύναμης σε περιβάλλον προσομοίωσης. Ο ελεγκτής επιτυγχάνει χαμηλό σφάλμα δύναμης σε μικρό χρόνο και διατηρεί την ακρίβεια με μοντελοποιημένες απώλειες. Στον πραγματικό μηχανισμό, πειραματικά, η εξάλειψη ενός μέρους της στατικής τριβής περιορίζει την επίδρασή της και προσφέρει πιο ομαλή και ρεαλιστική απτική αίσθηση.

This thesis examines force control on a planar five-bar haptic mechanism with two actuated and three passive joints. First, forward and inverse kinematic analyses are implemented, followed by differential kinematics with derivation of the Jacobian matrix and dynamic modeling. A PID position controller is then applied to assess the effect of the three gains. Next, human contact is modeled as a spring–damper pair, and force control is implemented in a simulation environment. The controller achieves low force error in a short time and maintains accuracy in the presence of modeled losses. On the physical mechanism, experiments show that partial compensation of static friction reduces its influence and yields a smoother and more realistic haptic sensation.

Contents

Chapter 1

Introduction

1.1 Definition of a Haptic Device

A haptic device is an electromechanical system that enables physical interaction between a human user and a computer or robotic system through the sense of touch. These devices incorporate sensors and actuators to detect user input—such as position, velocity, and force—and to provide tactile feedback in return. This bidirectional exchange allows users to feel and manipulate virtual or remote environments as though they were interacting with real physical objects.

Figure 1.1: 3D SYSTEMS Touch X Haptic Device

Haptic devices are used in a wide range of applications, including robotic surgery (Da Vinci robot - Figure 1.2), virtual and augmented reality (Senseglove Nova - Figure 1.3), and electronic gaming (Sony DualSense controller - Figure 1.4). In these contexts, haptic feedback enhances precision, immersion, and realism by allowing users to sense forces, textures, or resistances that would otherwise be imperceptible in a purely visual or auditory interface.

Figure 1.2: Da Vinci robot used in robotic-assisted surgery

Figure 1.3: Senseglove Nova for immersive virtual reality

Figure 1.4: Sony DualSense controller with advanced haptic feedback

1.2 Five-bar Linkage Device

The haptic device used in this thesis is based on a planar five-bar linkage mechanism (Figure 1.5). This parallel robotic structure consists of five rigid links and five revolute joints, two of which are actuated and three passive.

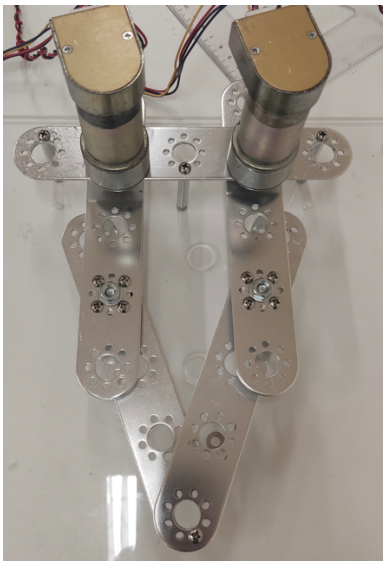


Figure 1.5: Five-bar Linkage Device

The two actuated joints, located at the fixed base points O and C, are driven by rotary motors and control the input angles θ_1 and θ_2 , respectively.

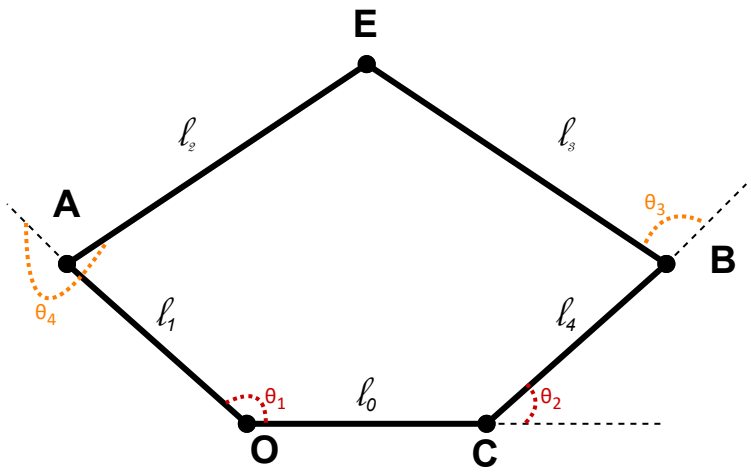


Figure 1.6: Five-bar Linkage Device Diagram.

In the context of haptic interaction, the user applies forces through the end-effector point E, while the system responds accordingly via the controlled motors at O and C. The geometric parameters of the system, such as the link lengths l_0 through l_4 and the motor placements, directly influence the workspace and mechanical behavior of the device.

1.3 System Hardware and Communication Setup

To enable the control and monitoring of the haptic device, a complete hardware architecture was implemented to facilitate communication between the mechanical system and a personal computer. This architecture includes all the necessary components for actuation, sensing, signal conversion, and power supply.

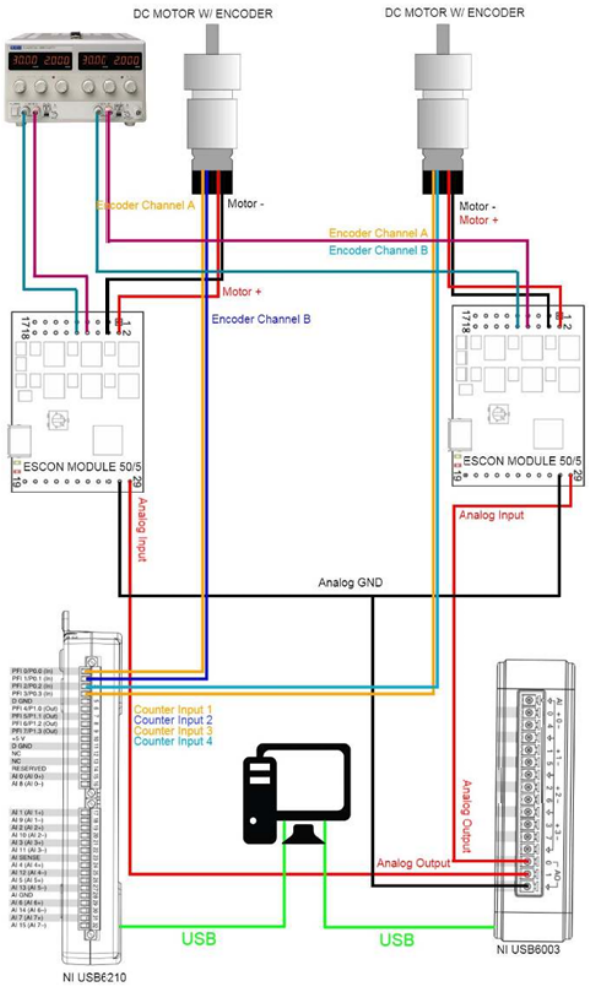


Figure 1.7: Hardware Architecture Diagram

The mechanism is driven by two DC motors with integrated encoders (Ametek Pittman GM8212D142), which provide real-time position feedback through pulse signals. These motors are controlled via ESCON 50/5 motor drivers (Figure 1.8), which are responsible for handling power delivery and executing movement commands.

Figure 1.8: Ametek Pittman GM8212D142

Figure 1.9: ESCON 50/5

Communication between the motor drivers and the computer is established using two National Instruments (NI) data acquisition devices. The NI USB-6210 (Figure 1.10) is used to read the encoder signals, translating position information into digital values accessible to the control software. The NI USB-6003 (Figure 1.11) handles the transmission of analog control signals from the PC to the motor drivers.

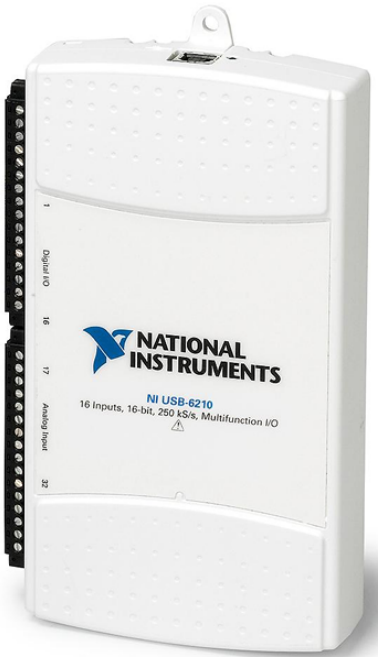


Figure 1.10: NI USB-6210

Figure 1.11: NI USB-6003

A power supply unit (EL302RD 3000) (Figure 1.12) is used to provide the required voltage and current levels to the motor drivers. The entire connection scheme is illustrated in the accompanying wiring diagram, showing how signals and power are distributed among components to ensure synchronized operation.

Figure 1.12: EL302RD 3000

This setup allows for the precise control of the haptic device and provides a reliable platform for developing and testing force control algorithms.

1.4 Communication with a Graphical Environment

The haptic device is integrated with a virtual simulation environment, enabling bidirectional communication between the physical and virtual domains. This interaction allows the user to receive force feedback that reflects the simulated contact with virtual objects, such as biological tissue in a surgical training scenario.

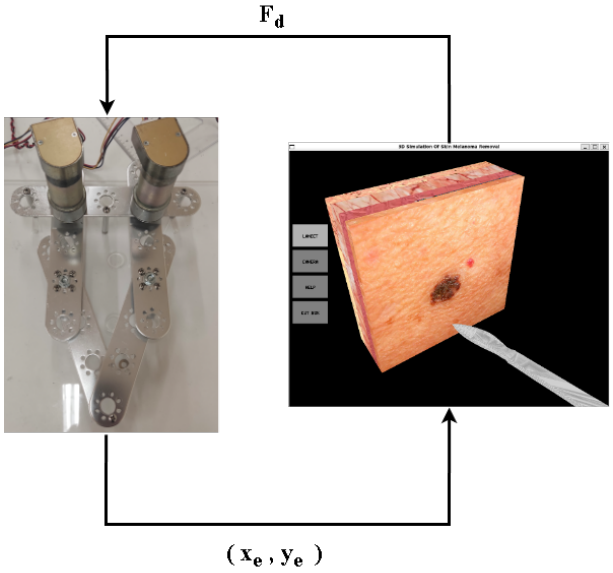


Figure 1.13: Communication diagram

The position of the end-effector (x_e , y_e) is continuously monitored and transmitted to the virtual environment via a network connection. This positional data is used to control the movement of a virtual scalpel, which interacts with the simulated tissue within the graphical environment.

In return, the graphical environment calculates the desired contact force F_d based on the interaction between the virtual scalpel and the tissue model, and sends it back to the physical system. This force is then applied to the user through the actuators of the haptic device, closing the loop of human-in-the-loop force feedback.

This architecture enables realistic and responsive interaction, essential for applications such as surgical training, remote manipulation, and immersive virtual experiences.

1.5 Objective and Challenges

The primary objective of this work is the development of a force controller for a haptic device, aiming to ensure realistic force sensation, accurate and stable dynamic response, and reduction of force losses, such as those caused by static friction.

In parallel, a position controller was developed primarily for familiarization purposes. This controller enabled initial experimentation with the five-bar linkage mechanism and facilitated an early understanding of its kinematic and dynamic behavior before proceeding to the more complex force control task.

Throughout the development process, several challenges were encountered. A significant limitation was the absence of a force sensor, which required the use of indirect estimation methods based on experimental force measurements with a dynamometer.

Additionally, replicating the complexity of human contact in a controlled system posed a substantial challenge, as it involves modeling the nuanced interaction between the human hand and the haptic mechanism in a realistic and responsive manner.

Chapter 2

Kinematic Analysis

In robotic systems, it is essential to determine both the position and velocity of the end-effector in relation to the motor angles and their respective velocities. This knowledge allows for accurate control of the robot's motion, ensuring that the end-effector reaches its desired position and follows the intended trajectory. Furthermore, calculating the Jacobian matrix is crucial for determining the required joint torques (τ) in static conditions. These torques are necessary for the motors to apply the forces needed to achieve the specified motions and maintain the stability of the end-effector under various loads, within the framework of implementing Force Control. This chapter provides an in-depth analysis of kinematic concepts, including forward and inverse kinematics, differential kinematics, and methods for validating both numerical and symbolic solutions.

2.1 Forward Kinematics

Forward kinematics provides the coordinates of the end-effector given that all characteristic link lengths and the angles of the driven links are known.

The process for locating the end-effector's coordinates is relatively simple. First, the coordinates of points A and B are determined through direct trigonometric calculations. Next, the circle equation is used to identify the intersection points of two circles centered at A and B, with respective radii of l_2 and l_3 (as shown in Figure 2.1).

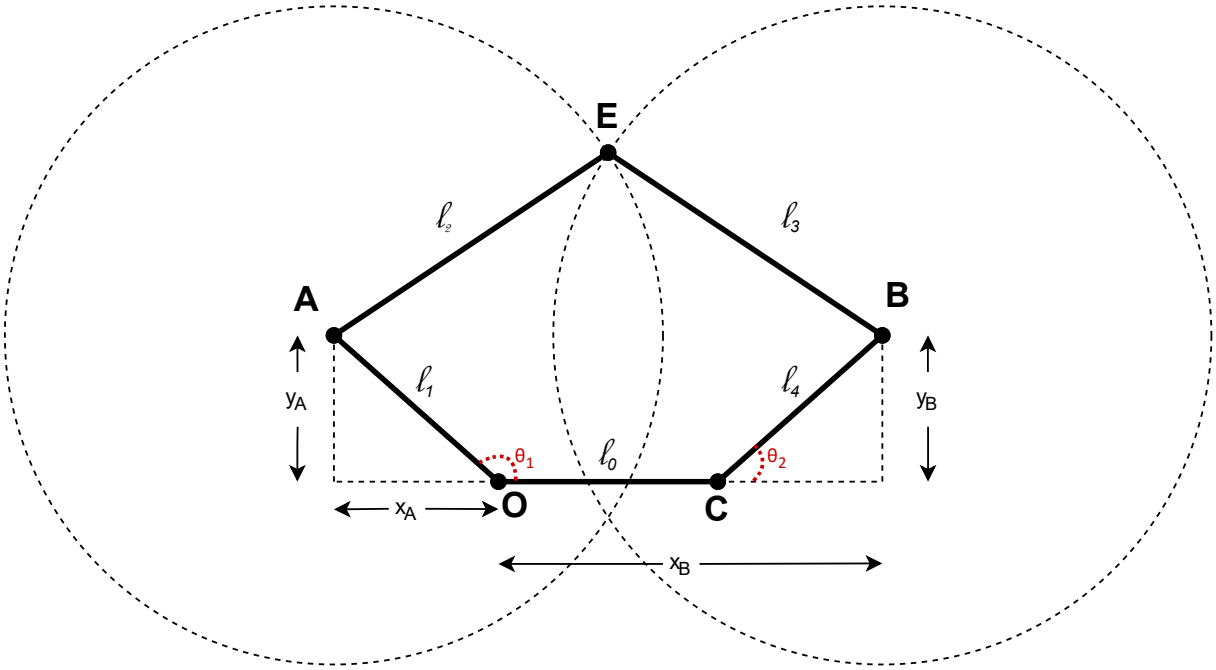


Figure 2.1: The positive solution of the intersection of the two circles provides the end-effector point.

Point O is considered the origin of the axes, and point E is the end-effector point. Using simple trigonometry, the coordinates of point A are derived as follows:

$$x_A = cos(\theta_1) * l_1 \quad \text{and} \quad y_A = sin(\theta_1) * l_1$$

In a similar way, the coordinates of point B are derived as follows:

$$x_B = l_0 + cos(\theta_2) * l_4 \quad \text{and} \quad y_B = sin(\theta_2) * l_4$$

Next, the above equations are used in the equations of the two circles.

For the circle centered at point A:

$$(x - x_A)^2 + (y - y_A)^2 = l_2^2 \Leftrightarrow$$

$$(x - cos(\theta_1) * l_1)^2 + (y - sin(\theta_1) * l_1)^2 = l_2^2$$

For the circle centered at point B:

$$(x - x_B)^2 + (y - y_B)^2 = l_2^2 \Leftrightarrow$$

$$(x - l_0 - cos(\theta_2) * l_4)^2 + (y - sin(\theta_2) * l_4)^2 = l_3^2$$

Applying the coordinates of the end-effector point to the equation of the circle centered at point A results in:

$$\begin{aligned} &(x_\epsilon - x_A)^2 + (y_\epsilon - y_A)^2 = l_2^2 \Leftrightarrow \\ &x_e^2 - 2 * x_\epsilon * x_A + x_A^2 + y_\epsilon^2 - 2 * y_\epsilon * y_A + y_A^2 - l_2^2 = 0 \end{aligned} \tag{2.1}$$

In a similar way, applying the coordinates of the end-effector point to the equation of the circle centered at point B results in:

$$\begin{aligned} &(x_\epsilon - x_B)^2 + (y_\epsilon - y_B)^2 = l_2^2 \Leftrightarrow \\ &x_e^2 - 2 * x_\epsilon * x_B + x_B^2 + y_\epsilon^2 - 2 * y_\epsilon * y_B + y_B^2 - l_3^2 = 0 \end{aligned} \tag{2.2}$$

Since the end-effector is the intersection point of the two circles, Eq. (2.1) and Eq. (2.2) are equal.

$(2.1) = (2.2) \Leftrightarrow$

$$x_\epsilon^2 - 2x_\epsilon x_A + x_A^2 + y_\epsilon^2 - 2y_\epsilon y_A + y_A^2 - l_2^2 = x_\epsilon^2 - 2x_\epsilon x_B + x_B^2 + y_e^2 - 2y_\epsilon y_B + y_B^2 - l_3^2 \Leftrightarrow$$

$$x_\epsilon = \frac{y_A - y_B}{x_B - x_A} * y_\epsilon + \frac{y_B^2 + x_B^2 - l_1^2 + l_2^2 - l_3^2}{2 * (x_B - x_A)} \Leftrightarrow$$

$$\boxed{x_\epsilon = a * y_\epsilon + b} \tag{2.3}$$

Subsequently, substituting Eq. (2.3) into Eq. (2.1) results in:

$$(a * y_\epsilon + b)^2 - 2 * x_A * (a * y_\epsilon + b) + x_A^2 + y_\epsilon^2 - 2 * y_\epsilon * y_A + y_A^2 - l_2^2 = 0 \Leftrightarrow$$

$$(a^2 + 1) * y_\epsilon^2 + (2 * a * b - 2 * a * x_A - 2 * y_A) * y_\epsilon + b^2 - 2 * b * x_A + l_1^2 - l_2^2 = 0 \Leftrightarrow$$

$$c * y_\epsilon^2 + d * y_\epsilon + e \Leftrightarrow$$

$$\boxed{y_\epsilon = \frac{-d \pm \sqrt{d^2 - 4 * c * e}}{2 * c}} \tag{2.4}$$

From the above expression for y_ϵ, the positive solution is retained.

Eq. (2.3) and Eq. (2.4) express the coordinates of end-effector point as a function of the motor angles θ_1 and θ_2.

2.2 Inverse Kinematics

Inverse kinematics determine the angles of the driven links when the characteristic link lengths and the coordinates of the end-effector are known.

The solution of Inverse kinematics can be obtained straightforwardly using the distance formula and the law of cosines.

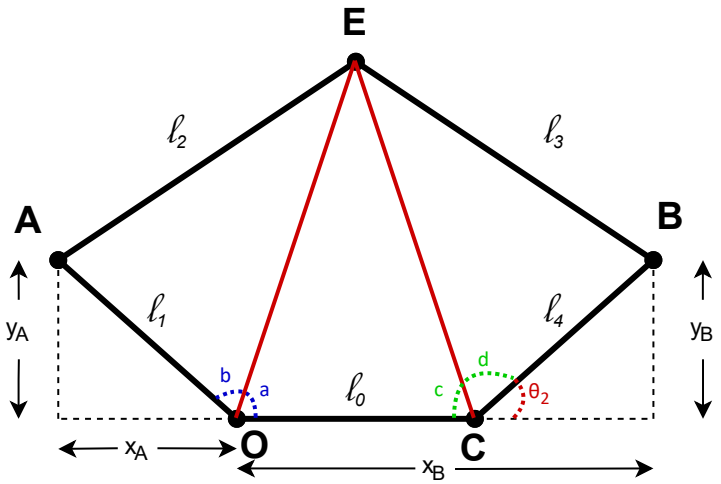


Figure 2.2: In the given diagram, two edges have been added that connect the end-effector point with points O and C.

Using the distance formula, the two new edges will be calculated.

$$OE = \sqrt{(x_\epsilon - x_O)^2 + (y_\epsilon - y_O)^2} \Leftrightarrow$$

$$OE^2 = x_\epsilon^2 + y_\epsilon^2$$

Similarly, for the other edge.

$$CE = \sqrt{(x_\epsilon - x_C)^2 + (y_\epsilon - y_C)^2} \Leftrightarrow$$

$$CE^2 = (x_\epsilon - l_0)^2 + y_\epsilon^2 \Leftrightarrow$$

$$CE^2 = x_\epsilon^2 - 2 * x_\epsilon * l_0 + l_0^2 + y_\epsilon^2$$

At this point, the law of cosines will be applied to triangle OEC in order to find angle a.

$$CE^2 = l_0^2 + OE^2 - 2 * l_0 * OE * cos(a) \Leftrightarrow$$

$$cos(a) = \frac{OE^2 - CE^2 + l_0^2}{2 * l_0 * OE} \Leftrightarrow$$

$$a = \arccos(\frac{x_\epsilon}{\sqrt{x_\epsilon^2 + y_\epsilon^2}})$$

Next, the law of cosines will be applied to triangle OEA in order to find angle b.

$$l_2^2 = l_1^2 + OE^2 - 2 * l_1 * OE * cos(b) \Leftrightarrow$$

$$cos(b) = \frac{l_1^2 - l_2^2 + x_\epsilon^2 + y_\epsilon^2}{2 * l_1 * \sqrt{x_\epsilon^2 + y_\epsilon^2}} \Leftrightarrow$$

$$b = \arccos(\frac{l_1^2 - l_2^2 + x_\epsilon^2 + y_\epsilon^2}{2 * l_1 * \sqrt{x_\epsilon^2 + y_\epsilon^2}})$$

Similarly to the first calculation, the law of cosines will be applied to triangle OEC again, but this time in order to find angle c.

$$OE^2 = CE^2 + l_0^2 - 2 * CE * l_0 * cos(c) \Leftrightarrow$$

$$cos(c) = \frac{2 * l_0^2 - 2 * x_\epsilon}{2 * \sqrt{(x_\epsilon - l_0)^2 + y_\epsilon^2} * l_0} \Leftrightarrow$$

$$c = \arccos(\frac{l_0 - x_\epsilon}{\sqrt{(x_\epsilon - l_0)^2 + y_\epsilon^2}})$$

Finally, the law of cosines will be applied to triangle CEB in order to find angle d.

$$l_3^2 = CE^2 + l_4^2 - 2 * CE * l_4 * cos(d) \Leftrightarrow$$

$$cos(d) = \frac{x_\epsilon^2 + y_\epsilon^2 + l_0^2 + l_4^2 - l_3^2 - 2 * x_\epsilon * l_0}{2 * \sqrt{(x_\epsilon - l_0)^2 + y_\epsilon^2} * l_4} \Leftrightarrow$$

$$d = \arccos(\frac{x_\epsilon^2 + y_\epsilon^2 + l_0^2 + l_4^2 - l_3^2 - 2 * x_\epsilon * l_0}{2 * \sqrt{(x_\epsilon - l_0)^2 + y_\epsilon^2} * l_4})$$

Having calculated the angles a, b, c, and d, the motor angles θ_1 and θ_2 are derived as follows:

$$\boxed{\theta_1 = a + b}$$

$$\boxed{\theta_2 = \pi \text{ - } (c + d)}$$

2.3 Validation of Inverse Kinematics

There is a very simple way to check if the Inverse Kinematics equations are correct.

Assuming that the motor angles θ_1 and θ_2 are 90°, Eq. (2.3) and Eq. (2.4) of Foward Kinematics are solved, and it follows that the coordinates of the end-effector point are:

$$x_\epsilon = 0.04\,\text{m} \quad \text{and} \quad y_\epsilon = 0.1931\,\text{m}$$

Now, the inverse process needs to be followed. The coordinates of the end-effector point are substituted into the Inverse Kinematics equations, and it follows that the motor angles θ_1 and θ_2 are:

$$\theta_1 = 1.572\,10\,\text{rad} = 90.07465^\circ$$

$$\theta_2 = 1.569\,49\,\text{rad} = 89.92513^\circ$$

It is observed that the motor angles θ_1 and θ_2 are approximately 90°, indicating that the Inverse Kinematics equations are correct. The small deviation from 90° is acceptable.

2.4 Representation of Passive Angles as a function of Motor Angles

The robotic system of the haptic device being analyzed contains two passive joints whose angles cannot be controlled by the two motors or measured by any sensor. These angles are θ_3 and θ_4. Therefore, they are unknown, and in order to be used in any calculation, they need to be expressed as functions of the known variables of the system, which include the links (l_0 - l_4) and the motor angles θ_1 and θ_1.

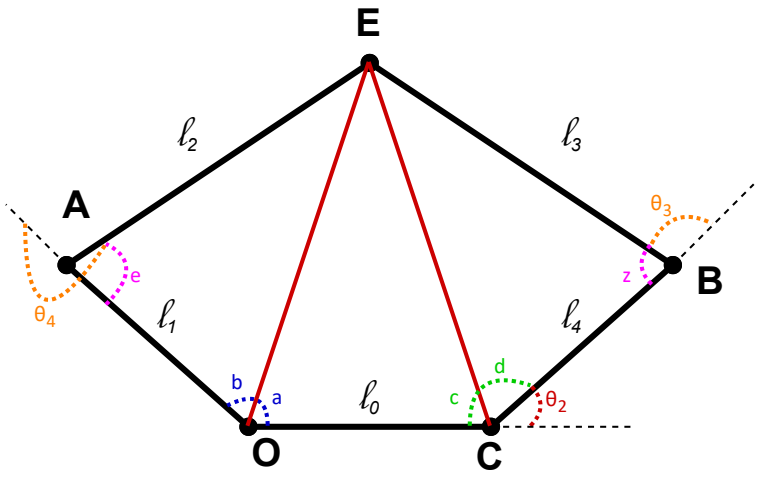


Figure 2.3: Points A and B represent the passive joints.

For the calculation of angles θ_3 and θ_4, it is easier to first calculate angles e and z.

The law of cosines will be applied to triangle OEA in order to find angle e.

$$OE^2 = l_1^2 + l_2^2 - 2 * l_1 * l_2 * cos(e) \Leftrightarrow$$

$$cos(e) = \frac{-x_\epsilon^2 - y_\epsilon^2 + l_1^2 + l_2^2}{2 * l_1 * l_2} \Leftrightarrow$$

$$e = \arccos(\frac{-x_\epsilon^2 - y_\epsilon^2 + l_1^2 + l_2^2}{2 * l_1 * l_2})$$

Next, the law of cosines will be applied to triangle CEB in order to find angle z.

$$CE^2 = l_3^2 + l_4^2 - 2 * l_3 * l_4 * cos(z) \Leftrightarrow$$

$$cos(z) = \frac{-x_\epsilon^2 - y_\epsilon^2 + 2 * x_\epsilon * l_0 - l_0^2 + l_3^2 + l_4^2}{2 * l_3 * l_4} \Leftrightarrow$$

$$z = \arccos(\frac{-x_\epsilon^2 - y_\epsilon^2 + 2 * x_\epsilon * l_0 - l_0^2 + l_3^2 + l_4^2}{2 * l_3 * l_4})$$

Having calculated the angles e and z, the passive angles θ_3 and θ_4 are derived as follows:

$$\boxed{\theta_3 = \pi \text{ - } z}$$

$$\boxed{\theta_4 = \pi + e}$$

2.5 Symbolic Solution of Differential Kinematics (Jacobian Matrix)

Differential kinematics provides the velocity of the end-effector point given that the velocities of the motor angles are known. Specifically, it is necessary to find the Jacobian matrix, which relates the velocity of the end-effector point to the velocities of the motor angles.

The straightforward solution for calculating the Jacobian matrix is to differentiate Eq. (2.3) and Eq. (2.4) with respect to θ_1 and θ_2 (numerical solution). However, since these equations are very complex and cannot be differentiated by hand, the Jacobian

matrix must be computed using another method.

Thus, one approach is to decompose the robotic system into two new separate systems, whose end-effector points will have the same velocity. For each system, the position equations of the end-effector are easily found through homogeneous transformations, and then they are differentiated to obtain the velocities, which will be equal.

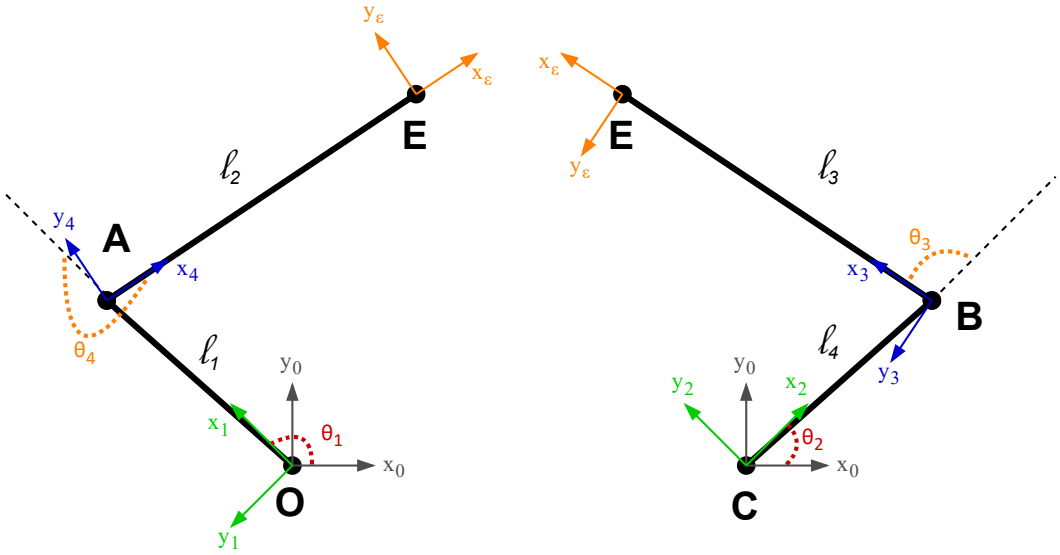


Figure 2.4: Two different robotic manipulators.

Through the combination of successive transformations T from coordinate frame 0 to coordinate frame E, the position of the end-effector point is calculated. Initially, this is performed for the left system.

$$ {}^{0}\mathbf{T}_{1} = \begin{bmatrix} \cos(\theta_1) & -\sin(\theta_1) & 0 & 0 \\ \sin(\theta_1) & \cos(\theta_1) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{1}\mathbf{T}_{4} = \begin{bmatrix} \cos(\theta_4) & -\sin(\theta_4) & 0 & l_1 \\ \sin(\theta_4) & \cos(\theta_4) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{4}\mathbf{T}_{\epsilon} = \begin{bmatrix} 1 & 0 & 0 & l_2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{0}\mathbf{T}_{4} = {}^{0}\mathbf{T}_{1}^{1}\mathbf{T}_{4} = \begin{bmatrix} \cos(\theta_1)\cos(\theta_4) - \sin(\theta_1)\sin(\theta_4) & -\cos(\theta_1)\sin(\theta_4) - \sin(\theta_1)\cos(\theta_4) & 0 & \cos(\theta_1)l_1 \\ \sin(\theta_1)\cos(\theta_4) + \cos(\theta_1)\sin(\theta_4) & -\sin(\theta_1)\sin(\theta_4) + \cos(\theta_1)\cos(\theta_4) & 0 & \sin(\theta_1)l_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{0}\mathbf{T}_{\epsilon} = {}^{0}\mathbf{T}_{4}^{4}\mathbf{T}_{\epsilon} = \begin{bmatrix} \cos(\theta_1+\theta_4) & -\sin(\theta_1+\theta_4) & 0 & (\cos(\theta_1)\cos(\theta_4)-\sin(\theta_1)\sin(\theta_4))l_2+\cos(\theta_1)l_1 \\ \sin(\theta_1+\theta_4) & \cos(\theta_1+\theta_4) & 0 & (\sin(\theta_1)\cos(\theta_4)+\cos(\theta_1)\sin(\theta_4))l_2+\sin(\theta_1)l_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

Where :

$$ \cos(\theta_1+\theta_4) = \cos(\theta_1)\cos(\theta_4) - \sin(\theta_1)\sin(\theta_4) $$

$$ \sin(\theta_1+\theta_4) = \sin(\theta_1)\cos(\theta_4) + \cos(\theta_1)\sin(\theta_4) $$

Differentiating the position equations of the end-effector point results in:

$$ \dot{x_\epsilon} = -l_2\sin(\theta_1)\cos(\theta_4)\dot{\theta_1} - l_2\cos(\theta_1)\sin(\theta_4)\dot{\theta_4} - l_2\cos(\theta_1)\sin(\theta_4)\dot{\theta_1} - l_2\sin(\theta_1)\cos(\theta_4)\dot{\theta_4} - l_1\sin(\theta_1)\dot{\theta_1} $$

$$ \dot{y_\epsilon} = l_2\cos(\theta_1)\sin(\theta_4)\dot{\theta_1} - l_2\sin(\theta_1)\sin(\theta_4)\dot{\theta_4} - l_2\sin(\theta_1)\sin(\theta_4)\dot{\theta_1} + l_2\cos(\theta_1)\cos(\theta_4)\dot{\theta_4} + l_1\cos(\theta_1)\dot{\theta_1} $$

Therefore, the Jacobian matrix is:

$$ {}^{0}\mathbf{J}_u = \begin{bmatrix} -l_1\sin(\theta_1) - l_2(\sin(\theta_1)\cos(\theta_4)+\cos(\theta_1)\sin(\theta_4)) & -l_2(\sin(\theta_1)\cos(\theta_4)+\cos(\theta_1)\sin(\theta_4)) \\ l_1\cos(\theta_1) + l_2(\cos(\theta_1)\cos(\theta_4)-\sin(\theta_1)\sin(\theta_4)) & l_2(\cos(\theta_1)\cos(\theta_4)-\sin(\theta_1)\sin(\theta_4)) \end{bmatrix} $$

For simplicity, the above matrix will be written as:

$$ {}^{0}\mathbf{J}_u = \begin{bmatrix} -\alpha_{00} & -\alpha_{01} \\ \alpha_{10} & \alpha_{11} \end{bmatrix} $$

Thus, the velocities of the end-effector point can be written as follows:

$$ \dot{x_\epsilon} = -\alpha_{00}\dot{\theta_1} - \alpha_{01}\dot{\theta_4} \qquad \dot{y_\epsilon} = \alpha_{10}\dot{\theta_1} + \alpha_{11}\dot{\theta_4} \tag{2.5} $$

Subsequently, the transformations for the right system are calculated.

$$
{}^{0}\mathbf{T}_{2} = \begin{bmatrix} \cos(\theta_2) & -\sin(\theta_2) & 0 & 0 \\ \sin(\theta_2) & \cos(\theta_2) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{2}\mathbf{T}_{3} = \begin{bmatrix} \cos(\theta_3) & -\sin(\theta_3) & 0 & l_4 \\ \sin(\theta_3) & \cos(\theta_3) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{3}\mathbf{T}_{\epsilon} = \begin{bmatrix} 1 & 0 & 0 & l_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
{}^{0}\mathbf{T}_{3} = {}^{0}\,\mathbf{T}_{2}^{2}\mathbf{T}_{3} = \begin{bmatrix} \cos(\theta_2)\cos(\theta_3) - \sin(\theta_2)\sin(\theta_3) & -\cos(\theta_2)\sin(\theta_3) - \sin(\theta_2)\cos(\theta_3) & 0 & \cos(\theta_2)l_4 \\ \sin(\theta_2)\cos(\theta_3) + \cos(\theta_2)\sin(\theta_3) & -\sin(\theta_2)\sin(\theta_3) + \cos(\theta_2)\cos(\theta_3) & 0 & \sin(\theta_2)l_4 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
{}^{0}\mathbf{T}_{\epsilon} = {}^{0}\,\mathbf{T}_{3}^{3}\mathbf{T}_{\epsilon} = \begin{bmatrix} \cos(\theta_2+\theta_3) & -\sin(\theta_2+\theta_3) & 0 & (\cos(\theta_2)\cos(\theta_3) - \sin(\theta_2)\sin(\theta_3))l_3 + \cos(\theta_2)l_4 \\ \sin(\theta_2+\theta_3) & \cos(\theta_2+\theta_3) & 0 & (\sin(\theta_2)\cos(\theta_3) + \cos(\theta_2)\sin(\theta_3))l_3 + \sin(\theta_2)l_4 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

Where :

$$\cos(\theta_2+\theta_3) = \cos(\theta_2)\cos(\theta_3) - \sin(\theta_2)\sin(\theta_3)$$

$$\sin(\theta_2+\theta_3) = \sin(\theta_2)\cos(\theta_3) + \cos(\theta_2)\sin(\theta_3)$$

Differentiating the position equations of the end-effector point results in:

$$\dot{x_\epsilon} = -l_3\sin(\theta_2)\cos(\theta_3)\dot{\theta_2} - l_3\cos(\theta_2)\sin(\theta_3)\dot{\theta_3} - l_3\cos(\theta_2)\sin(\theta_3)\dot{\theta_2} - l_3\sin(\theta_2)\cos(\theta_3)\dot{\theta_3} - l_4\sin(\theta_2)\dot{\theta_2}$$

$$\dot{y_\epsilon} = l_3\cos(\theta_2)\sin(\theta_3)\dot{\theta_2} - l_3\sin(\theta_2)\sin(\theta_3)\dot{\theta_3} - l_3\sin(\theta_2)\sin(\theta_3)\dot{\theta_2} + l_3\cos(\theta_2)\cos(\theta_3)\dot{\theta_3} + l_4\cos(\theta_2)\dot{\theta_2}$$

Therefore, the Jacobian matrix is:

$$ {}^{0}\mathbf{J}_u = \begin{bmatrix} -l_4\sin(\theta_2) - l_3(\sin(\theta_2)\cos(\theta_3) + \cos(\theta_2)\sin(\theta_3)) & -l_3(\sin(\theta_2)\cos(\theta_3) + \cos(\theta_2)\sin(\theta_3)) \\ l_4\cos(\theta_2) + l_3(\cos(\theta_2)\cos(\theta_3) - \sin(\theta_2)\sin(\theta_3)) & l_3(\cos(\theta_2)\cos(\theta_3) - \sin(\theta_2)\sin(\theta_3)) \end{bmatrix} $$

For simplicity, the above matrix will be written as:

$$ {}^{0}\mathbf{J}_u = \begin{bmatrix} -\beta_{00} & -\beta_{01} \\ \beta_{10} & \beta_{11} \end{bmatrix} $$

Thus, the velocities of the end-effector point can be written as follows:

$$ \dot{x_\epsilon} = -\beta_{00}\dot{\theta_2} - \beta_{01}\dot{\theta_3} \qquad \dot{y_\epsilon} = \beta_{10}\dot{\theta_2} + \beta_{11}\dot{\theta_3} \tag{2.6} $$

At this point, the velocities $\dot{\theta_3}$ and $\dot{\theta_4}$ must be eliminated from the velocity equations of the end-effector point, because, as previously mentioned, these angles cannot be controlled by the motors.

From the relation of $\dot{x_\epsilon}$ in Eq. (2.5), solving for $\dot{\theta_4}$ results in:

$$ \dot{\theta_4} = -\frac{\dot{x_\epsilon}}{\alpha_{01}} - \frac{\alpha_{00}\dot{\theta_1}}{\alpha_{01}} $$

Next, substituting the expression for $\dot{\theta_4}$ into relation of $\dot{y_\epsilon}$ in Eq. (2.5) results in:

$$ \dot{y_\epsilon} = -\frac{\alpha_{11}}{\alpha_{01}}\dot{x_\epsilon} + \frac{\alpha_{10}\alpha_{01} - \alpha_{11}\alpha_{00}}{\alpha_{01}}\dot{\theta_1} \tag{2.7} $$

The same process is followed for Eq. (2.6)

$$ \dot{\theta_3} = -\frac{\dot{x_\epsilon}}{\beta_{01}} - \frac{\beta_{00}\dot{\theta_2}}{\beta_{01}} $$

$$ \dot{y_\epsilon} = -\frac{\beta_{11}}{\beta_{01}}\dot{x_\epsilon} + \frac{\beta_{10}\beta_{01} - \beta_{11}\beta_{00}}{\beta_{01}}\dot{\theta_2} \tag{2.8} $$

At this point, Eq. (2.7) and Eq. (2.8) are set equal to each other because, as previously mentioned, the end-effector points of the two systems have the same velocities.

$$(2.7) = (2.8) \Leftrightarrow$$

$$-\frac{\alpha_{11}}{\alpha_{01}}\dot{x_\epsilon} + \frac{\alpha_{10}\alpha_{01} - \alpha_{11}\alpha_{00}}{\alpha_{01}}\dot{\theta_1} = -\frac{\beta_{11}}{\beta_{01}}\dot{x_\epsilon} + \frac{\beta_{10}\beta_{01} - \beta_{11}\beta_{00}}{\beta_{01}}\dot{\theta_2} \Leftrightarrow$$

$$\boxed{\dot{x_\epsilon} = -\frac{\beta_{01}(\alpha_{10}\alpha_{01}-\alpha_{11}\alpha_{00})}{\beta_{01}\alpha_{01}-\alpha_{11}\beta_{01}}\dot{\theta_1} + \frac{\alpha_{01}(\beta_{10}\beta_{01}-\beta_{11}\beta_{00})}{\beta_{11}\alpha_{01}-\alpha_{11}\beta_{01}}\dot{\theta_2}} \tag{2.9}$$

Substituting Eq. (2.9) into Eq. (2.7) results in:

$$\boxed{\dot{y_\epsilon} = (\frac{\alpha_{10}\alpha_{01}-\alpha_{11}\alpha_{00}}{\alpha_{01}} + \frac{\alpha_{11}\beta_{01}(\alpha_{10}\alpha_{01}-\alpha_{11}\alpha_{00})}{\alpha_{01}(\beta_{11}\alpha_{01}-\alpha_{11}\beta_{01})})\dot{\theta_1} - \frac{\alpha_{11}(\beta_{10}\beta_{01}-\beta_{11}\beta_{00})}{\beta_{11}\alpha_{01}-\alpha_{11}\beta_{01}}\dot{\theta_2}} \tag{2.10}$$

Eq. (2.9) and Eq. (2.10) represent the symbolic solutions of the Differential Kinematics. Thus, the resulting Jacobian matrix is:

$${}^0\mathbf{J}_u = \begin{bmatrix} \gamma_{00} & \gamma_{01} \\ \gamma_{10} & \gamma_{11} \end{bmatrix}$$

$$\gamma_{00} = -\frac{\beta_{01}(\alpha_{10}\alpha_{01} - \alpha_{11}\alpha_{00})}{\beta_{01}\alpha_{01} - \alpha_{11}\beta_{01}} \qquad \gamma_{01} = \frac{\alpha_{01}(\beta_{10}\beta_{01} - \beta_{11}\beta_{00})}{\beta_{11}\alpha_{01} - \alpha_{11}\beta_{01}}$$

$$\gamma_{10} = (\frac{\alpha_{10}\alpha_{01} - \alpha_{11}\alpha_{00}}{\alpha_{01}} + \frac{\alpha_{11}\beta_{01}(\alpha_{10}\alpha_{01} - \alpha_{11}\alpha_{00})}{\alpha_{01}(\beta_{11}\alpha_{01} - \alpha_{11}\beta_{01})}) \qquad \gamma_{11} = -\frac{\alpha_{11}(\beta_{10}\beta_{01} - \beta_{11}\beta_{00})}{\beta_{11}\alpha_{01} - \alpha_{11}\beta_{01}}$$

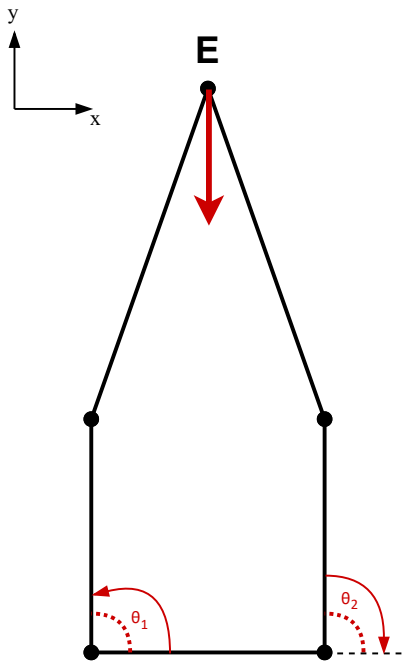
2.6 Validation of Kinematics and comparison of Numerical and Symbolic Solutions

To verify the correctness of the derived equations for the velocity of the end-effector point, two simple tests will be performed. In these tests, equal positive or negative velocity magnitudes will be applied to the two motor angles, and it will be checked, based on logic, whether the end-effector point moves in the correct direction.

Additionally, these tests will verify whether the symbolic solution of Differential Kinematics aligns with numerical solution.

Test 1:

Expected movement of the end-effector point in the negative direction of the y-axis.



For this test, the motor angles will be set to 90°. Additionally, a positive velocity will be assigned to angle θ_1, while a negative velocity of the same magnitude will be assigned to angle θ_2. These values are provided to the appropriate variables of the two Python programs presented earlier ((Symbolic solution) - (Numerical solution)).
The desired outcome, based on logic, is for the end-effector point to acquire a negative velocity along the y-axis, meaning it moves in the negative direction, while its velocity along the x-axis remains zero.

Angle	Position	Velocity
θ_1	90°	0.1745329252 rad/sec = 10 °/sec
θ_2	90°	- 0.1745329252 rad/sec = - 10 °/sec

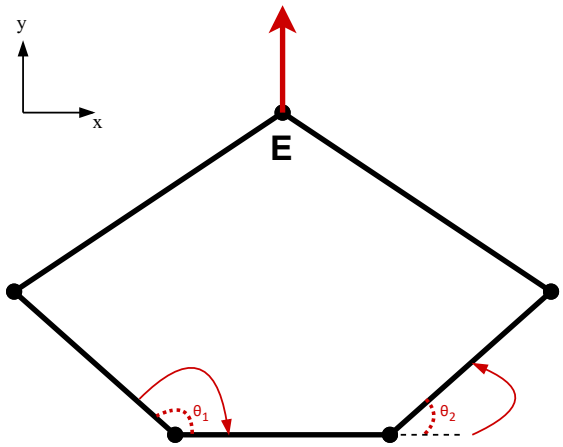
The results of the two programs are as follows:

Solution	Velocity along the x-axis	Velocity along the y-axis
Symbolic	- $0.20154 * 10^{-12}$ rad/sec $\approx$ 0 °/sec	- 0.0049365 rad/sec = - 0.282843 °/sec
Numerical	- $0.20153 * 10^{-12}$ rad/sec $\approx$ 0 °/sec	- 0.0049365 rad/sec = - 0.282843 °/sec

Initially, as observed, the two solutions align in terms of velocity values. Furthermore, as expected, the velocity along the y-axis is negative while along the x-axis it is approximately zero, confirming the correctness of the solutions.

Test 2:

Expected movement of the end-effector point in the positive direction of the y-axis.



For this test, motor angle θ_1 will be set to 135° and θ_2 will be set to 45°. Additionally, a positive velocity will be assigned to angle θ_2, while a negative velocity of the same magnitude will be assigned to angle θ_1. These values are provided to the appropriate variables of the two Python programs presented earlier ((Symbolic solution) - (Numerical solution)).

The desired outcome, based on logic, is for the end-effector point to acquire a positive velocity along the y-axis, meaning it moves in the positive direction, while its velocity along the x-axis remains zero.

Angle	Position	Velocity
θ_1	135°	- 0.6981317008 rad/sec = - 40 °/sec
θ_2	45°	0.6981317008 rad/sec = 40 °/sec

The results of the two programs are as follows:

Solution	**Velocity along the x-axis**	**Velocity along the y-axis**
Symbolic	- $0.3000100 * 10^{-12}$ rad/sec $\approx$ 0 °/sec	0.093028 rad/sec = 5.330112 °/sec
Numerical	- $0.3000030 * 10^{-12}$ rad/sec $\approx$ 0 °/sec	0.093028 rad/sec = 5.330112 °/sec

Initially, as observed, the two solutions align again in terms of velocity values. Furthermore, as expected, the velocity along the y-axis is positive while along the x-axis it is approximately zero, confirming the correctness of the solutions.

Chapter 3

PID Position Controller

A Proportional-Integral-Derivative (PID) position controller is a widely used control mechanism in robotic systems to accurately regulate the position of an end-effector. In this case, the objective of the PID controller is to drive the end-effector (point E) (Section 2.1) towards a desired target position while minimizing the position error. The controller continuously adjusts the actuator torques based on the current error, its rate of change, and its accumulated history, ensuring a stable and controlled motion.

This chapter examines the effect of each individual PID gain (Kp, Ki, and Kd) on the system's behavior. Specifically, by keeping two parameters constant while varying one, the impact of each gain on system performance will be studied separately. The primary focus is not to achieve the shortest possible movement time but to ensure a smooth and controlled motion trajectory.

To analyze the influence of the controller parameters, position and error plots will be presented in both the Cartesian space (describing the end-effector's position) and the actuator space (representing the motor angles). These plots provide valuable insights into how different gain values affect the system's response and overall motion stability.

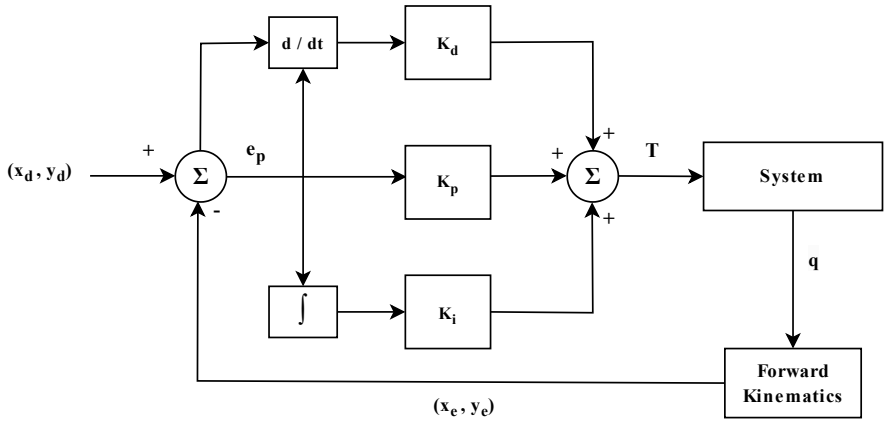


Figure 3.1: PID Position Controller Diagram

3.1 Calculation of the Desired Motor Angles

Through the PID Controller, appropriate torques will be applied to the two motors to minimize the error, which is defined as the difference between the desired motor angles (formed by the joints controlled by the motors when the end-effector reaches and stabilizes at the target position) and the current motor angles at any given moment.

Therefore, given a desired final position of the end-effector, the corresponding desired motor angles must first be calculated. Since it is desirable to restrict the motion of the end-effector within the boundaries of the Cartesian space, ensuring that no unreachable target positions are assigned, the two following steps are followed.

Step 1: Find the final position (x , y coordinates) of end-effector point.

Regardless of the way the mechanism is used, the initial motor angles are set to 90 degrees, as these angles allow the joints to be positioned visually, without the need for any trigonometric tool.

Additionally, using forward kinematics (Section 2.1), given the known motor angles, the initial position of the end-effector can also be determined.

Therefore, the first step is to manually position the mechanism in its initial configuration and then move the end-effector to any desired position. Once it reaches and stabilizes at that position, its desired final position is computed and using sensors and forward kinematics.

Step 2: Find the desired motor angles.

The second step is to input the coordinates of the previously computed desired final position into the inverse kinematics equations (Section 2.2), thereby obtaining the desired motor angles.

Finally, some essential aspects that must be understood for the proper interpretation of the graphical representations presented in the following sections are as follows:

Initial position of end-effector point : $(x_0 \ , \ y_0) = (0.0400 \text{ m} \ , \ 0.1931 \text{ m})$

Desired position of end-effector point : $(x_d \ , \ y_d) = (0.0400 \text{ m} \ , \ 0.0731 \text{ m})$

Initial position error of end-effector point : $(error_x \ , \ error_y) = (0 \text{ m} \ , \ 0.12 \text{ m})$

Initial position of motor angles : $(theta1_0 \ , \ theta2_0) = (90^\circ \ , \ 90^\circ)$

Desired position of motor angles : $(theta1_d \ , \ theta2_d) = (155.86^\circ \ , \ 24.14^\circ)$

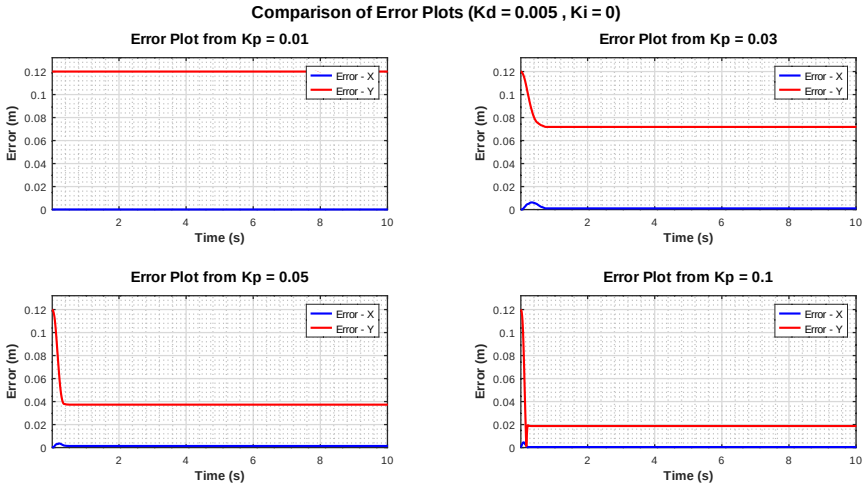
Initial position error of motor angles : $(error_1 \ , \ error_2) = (65.86^\circ \ , \ 65.86^\circ)$

3.2 Analysis of Kp Gain

The proportional gain (Kp) is a fundamental component of the PID controller, directly influencing how the system responds to position errors. In the context of robotic motion control, Kp determines how forcefully the controller corrects deviations from the desired trajectory. A higher Kp results in stronger corrections, reducing the final position error, but it may also introduce oscillations or instability if not properly tuned.

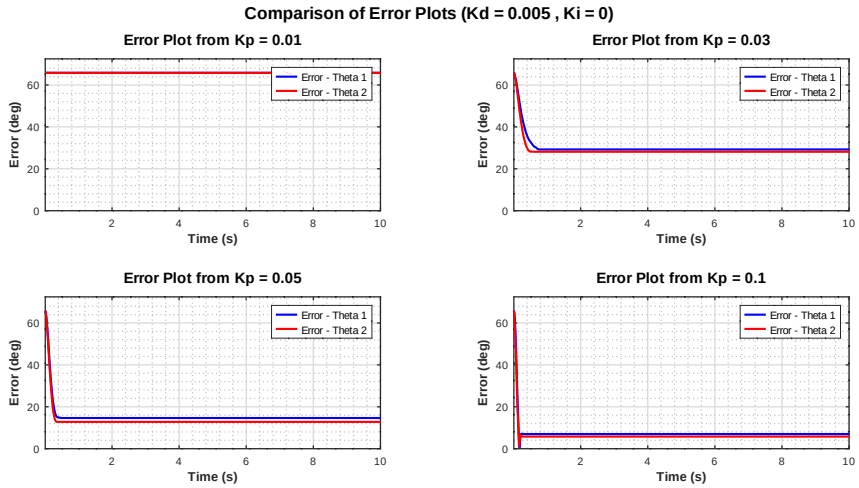
This section examines the impact of different Kp values on system performance. By keeping Ki and Kd constant, the response of the system is analyzed for varying proportional gain values, focusing on key performance metrics such as position accuracy, error convergence, and stability. The analysis is supported by error, position, and velocity plots, which illustrate the relationship between Kp and system behavior.

Graphical Representation of Position Error of End-Effector Point



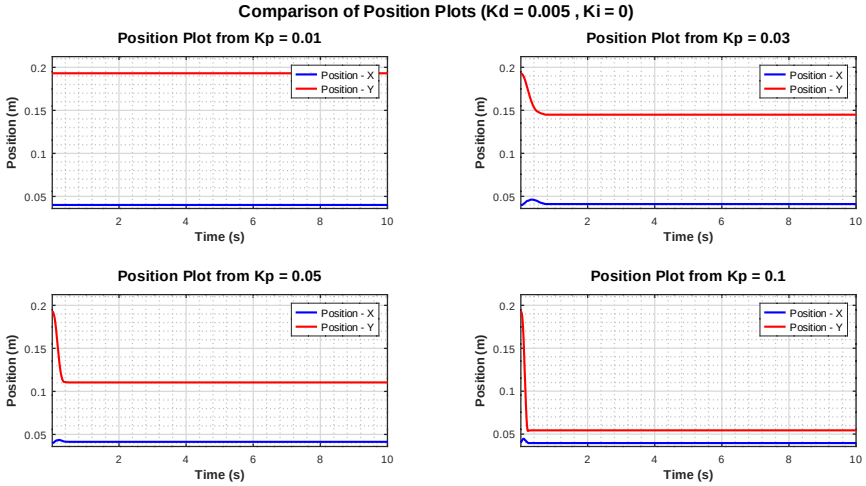
Based on the above graphical representations, it becomes evident that as Kp increases, the position error of end-effector decreases, allowing the system to reach the desired position more accurately and quickly. For low Kp values (e.g., 0.01), the error remains high, while higher values (e.g., 0.1) significantly reduce the steady-state error, particularly along the y-axis. The system responds faster with increasing Kp, however, excessive Kp values could introduce overshoot or oscillations, necessitating additional tuning, particularly of Kd, to ensure smoother convergence without instability.

Graphical Representation of Position Error of Motor Angles



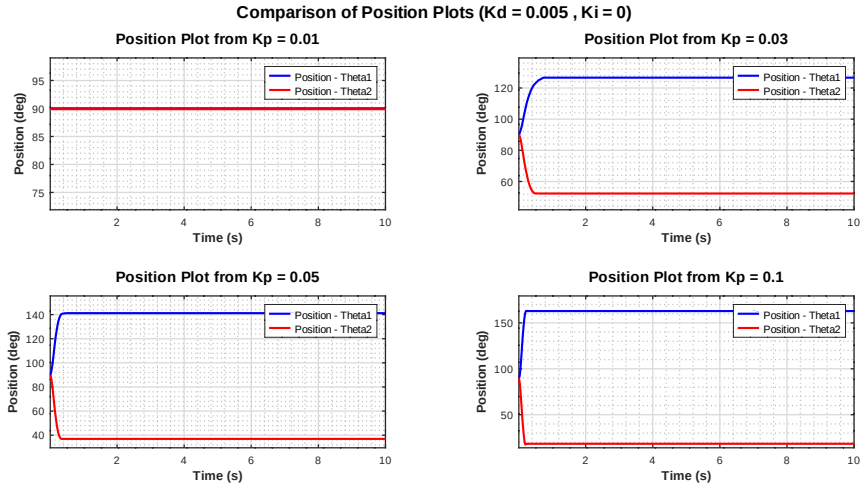
Similar to the position error of end-effector point, an increase in the Kp gain reduces the position error of the motor angles, leading to improved alignment with the desired angles. For low Kp values (e.g., 0.01), the error remains relatively high, indicating that the motors take longer to adjust their positions. Conversely, as Kp increases (e.g., 0.1), the system exhibits a significantly lower steady-state error, allowing motor angles to converge more efficiently to their target values.

Graphical Representation of Position of End-Effector Point



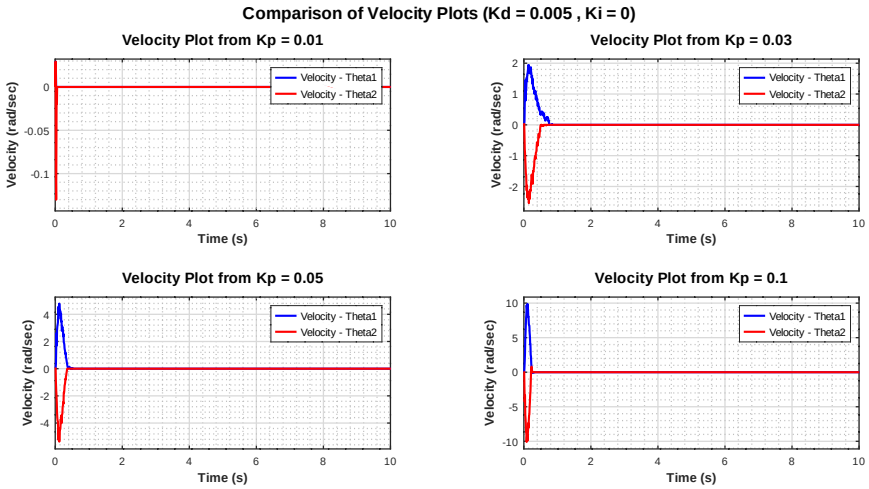
From the graphical representations above, it is clear that as Kp increases, the end-effector point reaches the desired position more rapidly and with greater accuracy. When Kp is low (e.g., 0.01), the system responds sluggishly, causing the end-effector to remain distant from its target position for a prolonged period. In contrast, as Kp increases (e.g., 0.03, 0.05, 0.1), the system achieves faster convergence toward the target position, particularly along the y-axis, where a significant reduction in settling time is observed. Moreover, for higher Kp values, the system stabilizes at the desired position much earlier, indicating enhanced response performance.

Graphical Representation of Position of Motor Angles



Based on the above graphical representations, it is evident that as Kp increases, motor angles converge more quickly to their desired values. For low Kp values (e.g., 0.01), motor angles remain close to their initial positions for a prolonged period, indicating a slow response. As Kp increases (e.g., 0.03, 0.05, 0.1), the system exhibits a much faster transition from the initial angles (90° for both motors) to the desired ones (155.86° and 24.14°). With higher Kp, the motion becomes significantly faster and more precise, reducing the time required for stabilization.

Graphical Representation of Velocity of Motor Angles



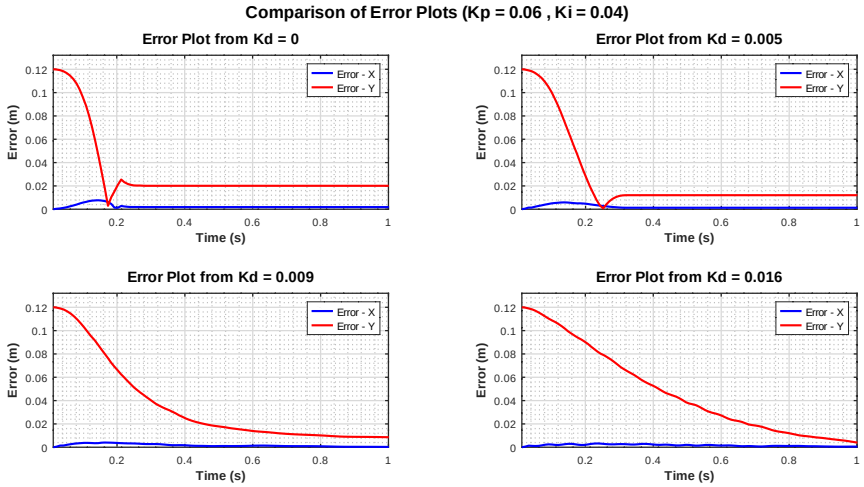
As Kp increases, motor velocities rise sharply, leading to a faster response. Low Kp values result in slow motor movement, while higher values (e.g., 0.1) cause higher peak velocities and quicker convergence. However, excessive Kp may introduce overshoot and instability, requiring proper Kd tuning for smoother motion.

3.3 Analysis of Kd Gain

The derivative gain (Kd) plays a crucial role in PID control, primarily improving the system's stability and response damping. In robotic motion control, Kd helps counteract rapid changes in error, reducing overshoot and minimizing oscillations that may arise from high Kp values. By predicting the system's future behavior based on the rate of error change, Kd enhances the smoothness of movement and prevents excessive corrections.

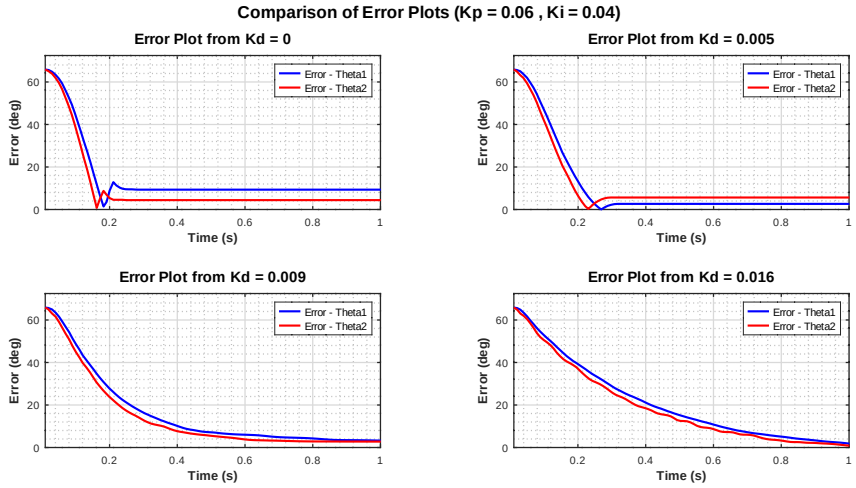
This section examines the effect of Kd on system performance while keeping Kp and Ki constant. The analysis explores how different Kd values influence error reduction, system damping, and velocity smoothness. To illustrate this, error, position, and velocity plots are presented, highlighting the relationship between Kd and the system behavior.

Graphical Representation of Position Error of End-Effector Point



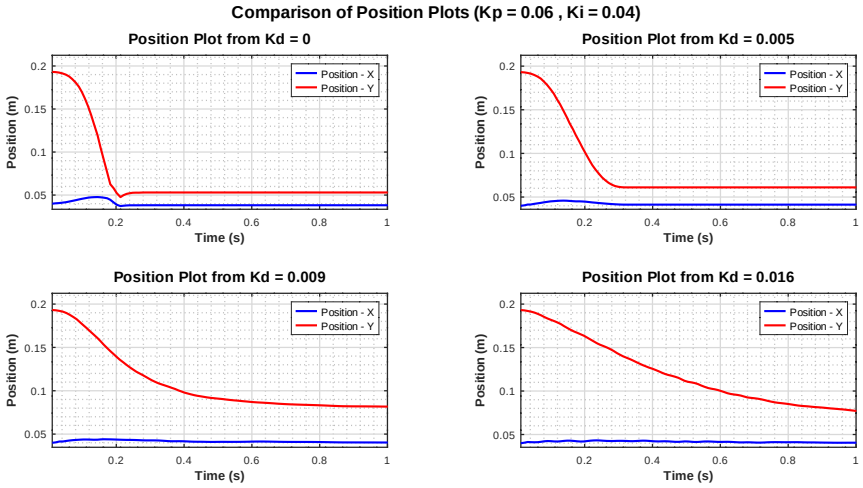
As the derivative gain (Kd) increases, the system exhibits improved damping, reducing overshoot and oscillations in the position error of the end-effector. At low Kd values (e.g., 0), significant oscillations and settling time are observed. Moderate Kd values (e.g., 0.005) effectively smooth the response, allowing the system to converge to the desired position faster. However, excessively high Kd values (e.g., 0.016) slow down the error reduction process, as the system becomes overly damped.

Graphical Representation of Position Error of Motor Angles



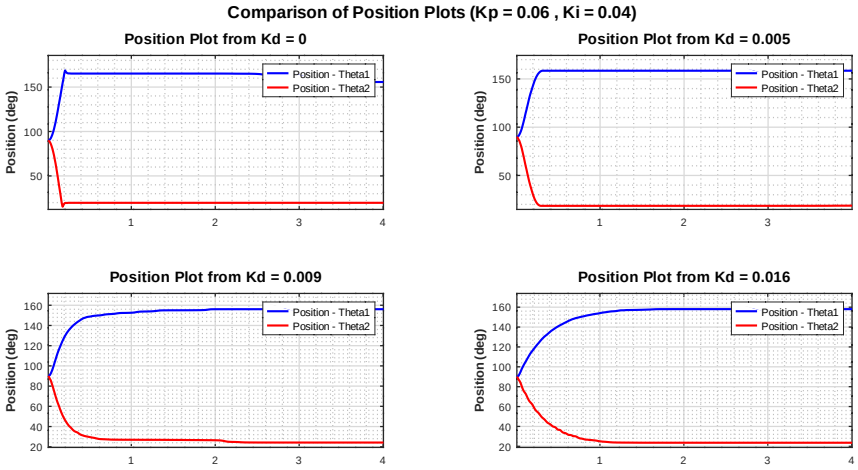
As the derivative gain (Kd) increases, the position error of motor angles experiences improved damping, effectively reducing oscillations and overshoot. When Kd = 0, significant fluctuations are observed before the angles reach stability. A moderate increase in Kd (e.g., 0.005) enhances the system's response, enabling faster convergence to the desired angles. However, excessively high values (e.g., 0.016) slow down the correction process, resulting in an overdamped system. Therefore, proper tuning of Kd is essential to achieve an optimal balance between error minimization and stability without introducing unnecessary delays.

Graphical Representation of Position of End-Effector Point



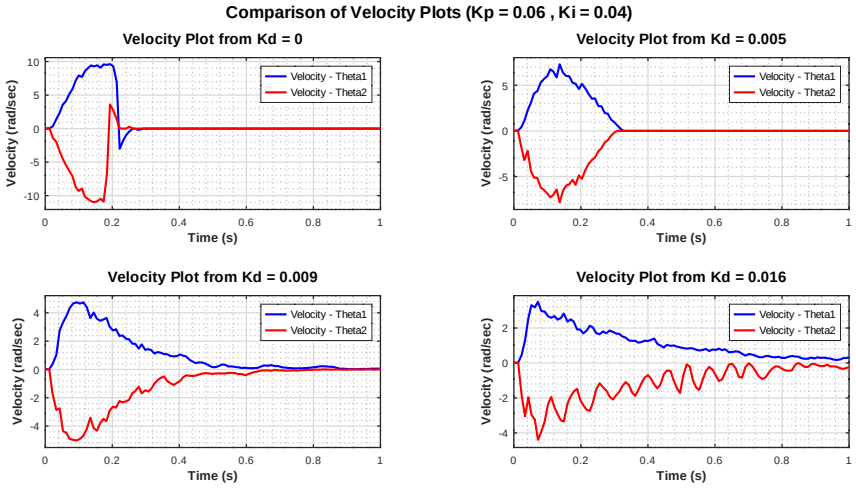
Similar to the previous cases, it is observed that as Kd increases, the movement of end-effector point becomes more stable, effectively reducing oscillations and overshoot. When Kd = 0, the system exhibits pronounced oscillations before reaching a steady state, whereas moderate values (e.g., 0.005) enhance stability and facilitate faster convergence. However, for higher values (e.g., 0.016), the system becomes overdamped, causing a slower approach to the desired position.

Graphical Representation of Position of Motor Angles



As the derivative gain (Kd) increases, motor angles exhibit improved damping, reducing oscillations and overshoot. When Kd is low (e.g., 0), motor angles reach the desired position with noticeable oscillations. In the other hand, with moderate Kd values (e.g., 0.005), the system stabilizes faster, achieving a smooth transition.

Graphical Representation of Velocity of Motor Angles



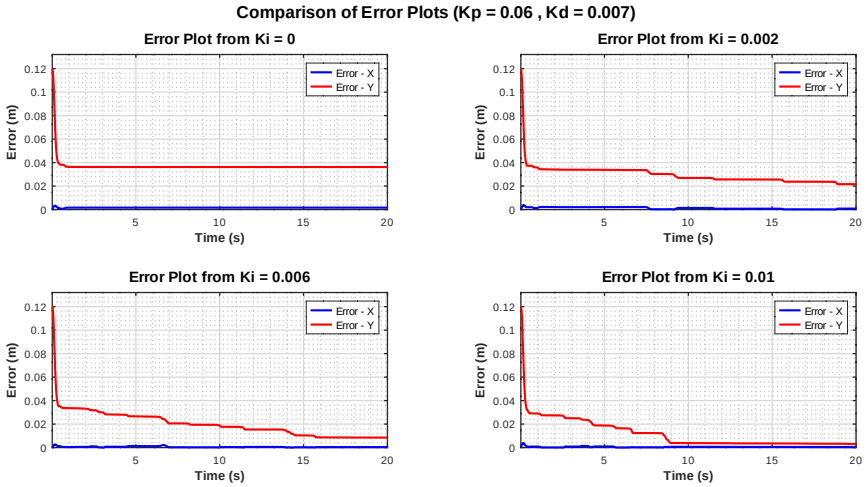
As the derivative gain (Kd) increases, the velocity response of motor angles becomes more stable, with reduced oscillations. When Kd is low (e.g., 0), the system exhibits high velocity peaks and oscillations, characteristic of an underdamped response. Moderate Kd values (e.g., 0.005) effectively suppress oscillations, resulting in a smoother velocity transition. However, excessively high Kd values (e.g., 0.016) lead to a slower response with small persistent oscillations, as the system becomes overdamped. Therefore, proper tuning of Kd is crucial to maintaining a balance between rapid response and smooth motion.

3.4 Analysis of Ki Gain

The integral gain (Ki) is a crucial component of the PID controller, primarily addressing steady-state errors that persist over time. In robotic motion control, Ki helps eliminate residual position errors by accumulating past errors and applying corrective actions based on their cumulative sum. This allows the system to achieve precise tracking of the desired trajectory, especially in the presence of external disturbances or model inaccuracies.

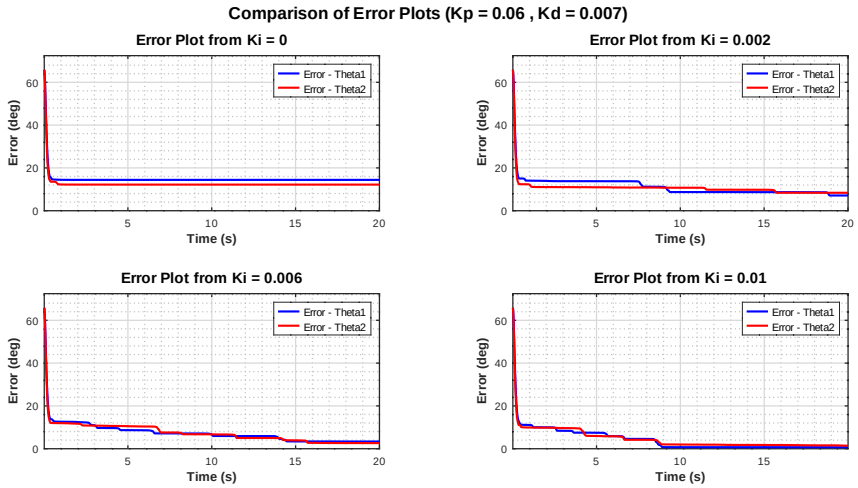
This section explores the impact of different Ki values on system performance. By keeping Kp and Kd constant, the system's response is analyzed for varying integral gain values, focusing on steady-state accuracy, error elimination, and stability. The analysis is supported by error, position, and velocity plots, providing insights into how Ki influences the system's ability to maintain the desired position over time.

Graphical Representation of Position Error of End-Effector Point



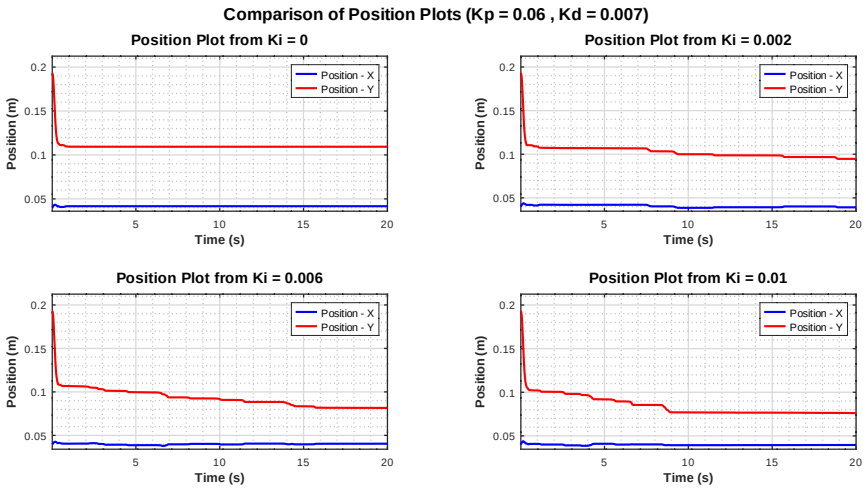
As the integral gain (Ki) increases, the steady-state error of end-effector position decreases, allowing the system to gradually correct accumulated deviations. At Ki = 0, a residual error remains, particularly along the y-axis. With moderate Ki values (e.g., 0.002), the system progressively reduces this error over time. Higher Ki values (e.g., 0.01) further improve error elimination but introduce slower convergence. Excessively high Ki may lead to instability or excessive overshooting, making precise tuning essential for maintaining balance between accuracy and stability.

Graphical Representation of Position Error of Motor Angles



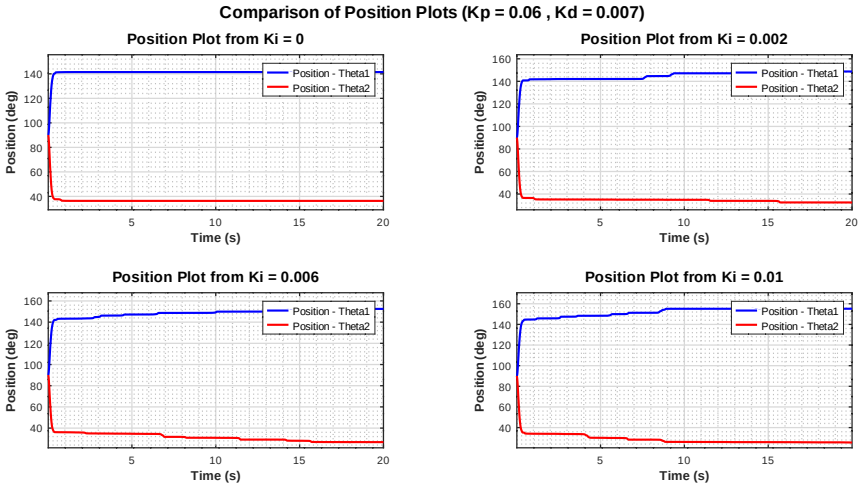
As Ki increases, the steady-state error in motor angles decreases, enabling the system to correct accumulated deviations. At Ki = 0, residual error prevents full convergence, while moderate values (e.g., 0.002) gradually reduce it. Higher Ki values (e.g., 0.01) improve correction but slow convergence.

Graphical Representation of Position of End-Effector Point



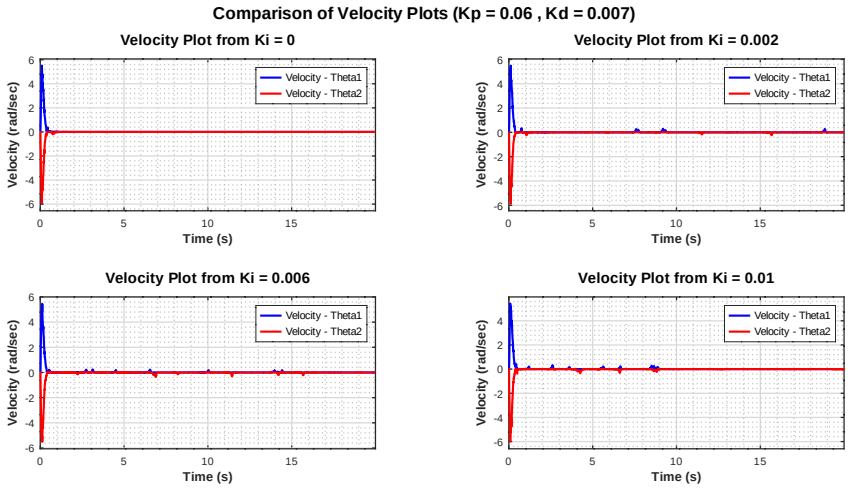
As the integral gain (Ki) increases, the position of end-effector gradually corrects accumulated deviations, reducing steady-state error. At Ki = 0, the system stabilizes but retains residual error. With moderate Ki values (e.g., 0.002), the position adjusts progressively, showing improved accuracy over time. Higher Ki values (e.g., 0.01) further enhance convergence to the desired position but slow the correction process.

Graphical Representation of Position of Motor Angles



As the integral gain (Ki) increases, motor angles gradually align more accurately with their desired values by compensating for accumulated errors. When Ki = 0, a steady-state error persists, preventing full correction. Moderate Ki values (e.g., 0.002) enhance the adjustment process over time, while higher values (e.g., 0.01) further improve convergence but slow stabilization. Excessively high Ki may introduce instability or oscillations, requiring careful tuning to balance accuracy and smooth motion.

Graphical Representation of Velocity of Motor Angles



As the integral gain (Ki) increases, the velocity response of motor angles becomes more stable and consistent over time. At Ki = 0, the velocity stabilizes quickly but leaves a steady-state error. With moderate Ki values (e.g., 0.002), small velocity adjustments occur to further correct deviations. Higher Ki values (e.g., 0.01) provide continuous error correction but slow down stabilization.

Chapter 4

Dynamics Analysis

Understanding the dynamic behavior of a robotic system is essential for developing effective control strategies, particularly in applications involving interaction with external forces, such as in haptic devices. This chapter presents a comprehensive dynamic analysis of the pantograph-based mechanism under study. By formulating the system's equations of motion using the Lagrangian approach, we identify the contributions of inertial, Coriolis, and centrifugal forces. The derived dynamic model serves as a foundation for implementing force-based control strategies and simulating realistic motion responses. The analysis considers both the active and passive joints of the mechanism, enabling an accurate representation of the system's response to motor torques and external user-applied forces.

It should be noted that the naming convention for the joint angles used in this chapter differs from that adopted in the previous chapters

4.1 Computation of the Dynamic equations

The above figure presents the dynamic model of the pantograph haptic mechanism, where all relevant geometric and kinematic quantities are illustrated. The mechanism consists of five rigid links labeled A, B, C, D and the base N, connected through revolute joints. Each link is associated with a local coordinate frame, and their orientation in space is described by a pair of unit vectors: a_1, a_2 for link A, b_1, b_2 for link B, c_1, c_2 for link C and d_1, d_2 for link D. The base coordinate system is defined by the orthonormal unit vectors n_1, n_2 which lie in the plane of motion.

The angular position of each link is represented by the generalized coordinates q_1, q_2, q_3, q_4, which describe the counterclockwise rotation of each link with respect to the base frame. The symbols m_A, m_B, m_C, m_D, denote the masses of the respective links, and the distances l_1, l_2, l_3, l_4, correspond to their lengths, while l_0 represents the fixed base distance between the two grounded joints.

The red dashed vectors shown in the diagram represent the position vectors of the centers of mass of the links with respect to the base frame. Specifically, the vectors r^{0A_0}, r^{0B_0}, r^{0C_0}, r^{0D_0}, point to the center of mass of links A, B, C and D respectively.These position vectors are fundamental for the derivation of the system's kinetic energy and subsequently for the formulation of the dynamic equations of motion.

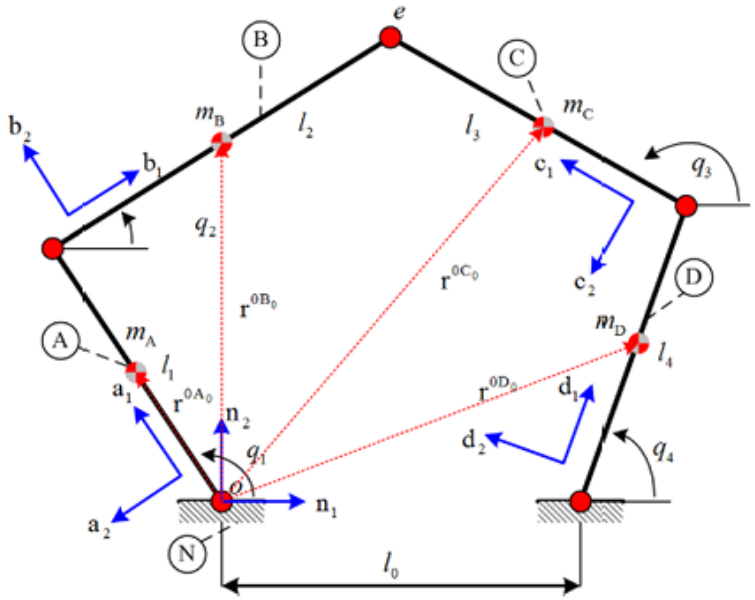


Figure 4.1: A new representation of the mechanism, using a different set of angle notations.

Step 1: Find the Position Vectors of the links centers of mass.

The position vector of the center of mass of link A is given by

$$\mathbf{r}^{0A_0} = \frac{l_1}{2}\mathbf{a_1} = \frac{l_1}{2}cos(q_1)\mathbf{n_1} + \frac{l_1}{2}sin(q_1)\mathbf{n_2} \tag{4.1}$$

The position vector of the center of mass of link B is given by

$$\mathbf{r}^{0B_0} = l_1\mathbf{a_1} + \frac{l_2}{2}\mathbf{b_1} \Leftrightarrow$$

$$\mathbf{r}^{0B_0} = l_1 cos(q1)\mathbf{n_1} + l_1 sin(q1)\mathbf{n_2} + \frac{l_2}{2}cos(q_2)\mathbf{n_1} + \frac{l_2}{2}sin(q_2)\mathbf{n_2} \Leftrightarrow \tag{4.2}$$

$$\mathbf{r}^{0B_0} = \left(l_1 cos(q1) + \frac{l_2}{2}cos(q_2)\right)\mathbf{n_1} + \left(l_1 sin(q1) + \frac{l_2}{2}sin(q_2)\right)\mathbf{n_2}$$

Similarly, the position vector of the center of mass of link D is give by

$$\mathbf{r}^{0D_0} = l_0\mathbf{n_1} + \frac{l_4}{2}\mathbf{d_1} \Leftrightarrow$$

$$\mathbf{r}^{0D_0} = l_0\mathbf{n_1} + \frac{l_4}{2}\left(cos(q_4)\mathbf{n_1} + sin(q_4)\mathbf{n_2}\right) \Leftrightarrow \tag{4.3}$$

$$\mathbf{r}^{0D_0} = \left(l_0 + \frac{l_4}{2}cos(q_4)\right)\mathbf{n_1} + \frac{l_4}{2}sin(q_4)\mathbf{n_2}$$

Finally, the position vector of the center of mass of link C is given by

$$\mathbf{r}^{0C_0} = l_0\mathbf{n_1} + l_4\mathbf{d_1} + \frac{l_3}{2}\mathbf{c_1} \Leftrightarrow$$

$$\mathbf{r}^{0C_0} = l_0 n_1 + l_4 cos(q_4)\mathbf{n_1} + l_4 sin(q_4)\mathbf{n_2} + \frac{l_3}{2}cos(q_3)\mathbf{n_1} + \frac{l_3}{2}sin(q_3)\mathbf{n_2} \Leftrightarrow \tag{4.4}$$

$$\mathbf{r}^{0C_0} = \left(l_0 + l_4 cos(q_4) + \frac{l_3}{2}cos(q_3)\right)\mathbf{n_1} + \left(l_4 sin(q_4) + \frac{l_3}{2}sin(q_3)\right)\mathbf{n_2}$$

Step 2: Compute the Velocities of the links centers of mass.

Now we calculate the velocity vector of the calculated vectors in the frame of reference N as follows:

$$
\begin{aligned}
{}^N\mathbf{v}^{A_0} &=^N \frac{d}{dt}\mathbf{r}^{0A_0} \Leftrightarrow \\
{}^N\mathbf{v}^{A_0} &= -\frac{l_1}{2}\dot{q}_1 \sin(q_1)\mathbf{n}_1 + \frac{l_1}{2}\dot{q}_1 \cos(q_1)\mathbf{n}_2
\end{aligned}
\tag{4.5}
$$

The velocity vector of the center of mass of link B is calculated using

$$
\begin{aligned}
{}^N\mathbf{v}^{B_0} &=^N \frac{d}{dt}\mathbf{r}^{0B_0} \Leftrightarrow \\
{}^N\mathbf{v}^{B_0} &= -\left(l_1\dot{q}_1 \sin(q_1) + \frac{l_2}{2}\dot{q}_2 \sin(q_2)\right)\mathbf{n}_1 + \left(l_1\dot{q}_1 \cos(q_1) + \frac{l_2}{2}\dot{q}_2 \cos(q_2)\right)\mathbf{n}_2
\end{aligned}
\tag{4.6}
$$

Similarly, the velocity vector of the center of mass of link D is given by

$$
\begin{aligned}
{}^N\mathbf{v}^{D_0} &=^N \frac{d}{dt}\mathbf{r}^{0D_0} \Leftrightarrow \\
{}^N\mathbf{v}^{D_0} &= -\left(\frac{l_4}{2}\dot{q}_4 \sin(q_4)\right)\mathbf{n}_1 + \left(\frac{l_4}{2}\dot{q}_4 \cos(q_4)\right)\mathbf{n}_2
\end{aligned}
\tag{4.7}
$$

Finally, the velocity vector of the center of mass of link C is calculated as follows:

$$
\begin{aligned}
{}^N\mathbf{v}^{C_0} &=^N \frac{d}{dt}\mathbf{r}^{0C_0} \Leftrightarrow \\
{}^N\mathbf{v}^{C_0} &= -\left(l_4\dot{q}_4 \sin(q_4) + \frac{l_3}{2}\dot{q}_3 \sin(q_3)\right)\mathbf{n}_1 + \left(l_4\dot{q}_4 \cos(q_4) + \frac{l_3}{2}\dot{q}_3 \cos(q_3)\right)\mathbf{n}_2
\end{aligned}
\tag{4.8}
$$

Step 3: Compute the total Kinetic Energy.

Link A is in rational motion only. Therefore, its kinetic energy (T_A) is calculated using

$$
T_A = \frac{1}{2} \cdot \frac{m_A l_1^2}{3} \cdot \dot{q}_1^2 \tag{4.9}
$$

Link D exhibits only rotational motion. Therefore, its kinetic energy (T_D) is calculated using

$$T_D = \frac{1}{2} \cdot \frac{m_D l_4^2}{3} \cdot \dot{q}_4^2 \tag{4.10}$$

The kinetic energy of Link B is due to translation and rotation and can be calculated as

$$\begin{aligned} T_B &= \frac{1}{2} I_{CB} \dot{q}_2^2 + \frac{1}{2} m_B \, {}^N\mathbf{v}^{B_0} \cdot {}^N\mathbf{v}^{B_0} \Leftrightarrow \\ T_B &= \frac{1}{2} \cdot \frac{m_B l_2^2}{12} \dot{q}_2^2 + \frac{1}{2} m_B \left(l_1^2 \dot{q}_1^2 + \frac{l_2^2}{4} \dot{q}_2^2 + \frac{l_1 l_2}{2} \dot{q}_1 \dot{q}_2 \cos(q_1 - q_2) \right) \end{aligned} \tag{4.11}$$

Similarly, the kinetic energy of Link C is due to translation and rotation and can be calculated as

$$\begin{aligned} T_C &= \frac{1}{2} I_{CC} \dot{q}_3^2 + \frac{1}{2} m_C \, {}^N\mathbf{v}^{C_0} \cdot {}^N\mathbf{v}^{C_0} \Leftrightarrow \\ T_C &= \frac{1}{2} \cdot \frac{m_C l_3^2}{12} \dot{q}_3^2 + \frac{1}{2} m_C \left(l_4^2 \dot{q}_4^2 + \frac{l_3^2}{4} \dot{q}_3^2 + \frac{l_3 l_4}{2} \dot{q}_3 \dot{q}_4 \cos(q_3 - q_4) \right) \end{aligned} \tag{4.12}$$

The total kinetic energy of five bar linkage haptic device is

$$T = T_A + T_B + T_C + T_D \tag{4.13}$$

Step 4: Compute the Euler–Lagrange Equations.

Finally, the equations of motion of the pantograph haptic device are

$$\frac{d}{dt} \left(\frac{\partial T}{\partial \dot{q}_i} \right) - \frac{\partial T}{\partial q_i} = Q_i \quad \text{for } i = 1, \ldots, 4 \tag{4.14}$$

where Q_i is the generalized force associated with the generalized coordinate i.

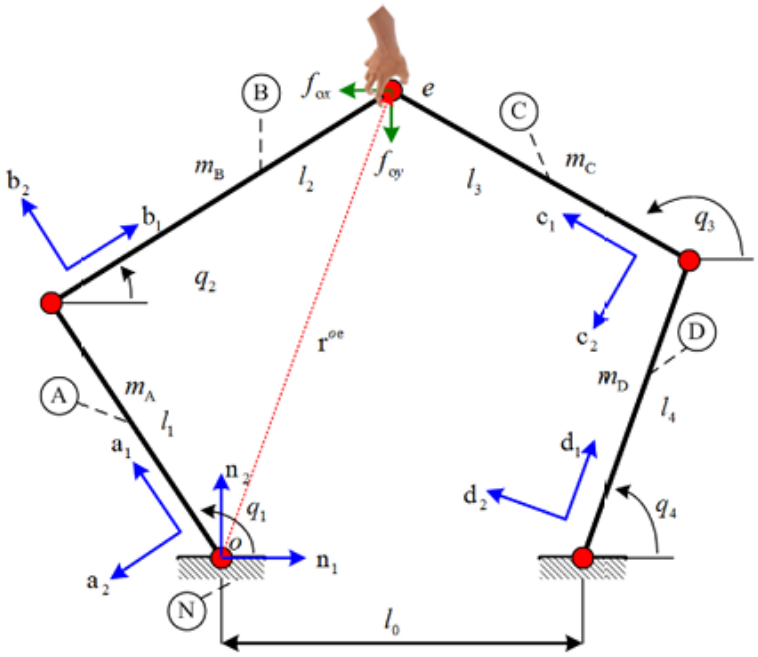


Figure 4.2: Representation of the mechanism with the presence of human hand.

Step 5: Computation of the Generalized Forces Q_i for the Joint Coordinates i.

The generalized force associated with q_1 is

$$Q_1 = \mathbf{F}_o \cdot \frac{\partial \mathbf{r}^{oe}}{\partial q_1} \Leftrightarrow$$
$$Q_1 = -f_{ox} l_1 \sin(q_1) + f_{oy} l_1 \cos(q_1) + T_1 \tag{4.15}$$

where F_o is the interaction force of the operator with the pantograph haptic device at a point e. Further, f_{ox} and f_{oy} are the components of F_o in the frame of reference, along n_1 and n_2 , respectively. T_1 is the control torque input exerted on the generalized coordinate q_1. The generalized force associated with q_2 is

$$Q_2 = \mathbf{F}_o \cdot \frac{\partial \mathbf{r}^{oe}}{\partial q_2} \Leftrightarrow$$
$$Q_2 = -f_{ox} l_2 \sin(q_2) + f_{oy} l_2 \cos(q_2) \tag{4.16}$$

The generalized force associated with q_3 is

$$\begin{aligned} Q_3 &= \mathbf{F}_o \cdot \frac{\partial \mathbf{r}^{oe}}{\partial q_3} \Leftrightarrow \\ Q_3 &= -f_{ox} l_3 \sin(q_3) + f_{oy} l_3 \cos(q_3) \end{aligned} \tag{4.17}$$

The generalized force associated with q_4 is

$$\begin{aligned} Q_4 &= \mathbf{F}_o \cdot \frac{\partial \mathbf{r}^{oe}}{\partial q_4} \Leftrightarrow \\ Q_4 &= -f_{ox} l_4 \sin(q_4) + f_{oy} l_4 \cos(q_4) + T_2 \end{aligned} \tag{4.18}$$

Step 6: Equating the results from steps 4 and 5.

$$\begin{aligned} &\left(\frac{m_A}{3} + m_B\right) l_1^2 \ddot{q}_1 + \frac{m_B l_1 l_2}{4} \ddot{q}_2 \cos(q_1 - q_2) - \frac{m_B l_1 l_2}{4} (\dot{q}_1 - \dot{q}_2) \dot{q}_2 \sin(q_1 - q_2) \\ &+ \frac{m_B l_1 l_2}{4} \dot{q}_1 \dot{q}_2 \sin(q_1 - q_2) = -f_{ox} l_1 \sin(q_1) + f_{oy} l_1 \cos(q_1) + T_1, \end{aligned} \tag{4.19}$$

$$\begin{aligned} &\frac{m_B l_2^2}{3} \ddot{q}_2 + \frac{m_B l_1 l_2}{4} \ddot{q}_1 \cos(q_1 - q_2) - \frac{m_B l_1 l_2}{4} (\dot{q}_1 - \dot{q}_2) \dot{q}_1 \sin(q_1 - q_2) \\ &- \frac{m_B l_1 l_2}{4} \dot{q}_1 \dot{q}_2 \sin(q_1 - q_2) = -f_{ox} l_2 \sin(q_2) + f_{oy} l_2 \cos(q_2), \end{aligned} \tag{4.20}$$

$$\begin{aligned} &\frac{m_C l_3^2}{3} \ddot{q}_3 + \frac{m_C l_3 l_4}{4} \ddot{q}_4 \cos(q_3 - q_4) - \frac{m_C l_3 l_4}{4} (\dot{q}_3 - \dot{q}_4) \dot{q}_4 \sin(q_3 - q_4) \\ &- \frac{m_C l_3 l_4}{4} \dot{q}_3 \dot{q}_4 \sin(q_3 - q_4) = -f_{ox} l_3 \sin(q_3) + f_{oy} l_3 \cos(q_3), \end{aligned} \tag{4.21}$$

$$\begin{aligned} &\left(\frac{m_D}{3} + m_C\right) l_4^2 \ddot{q}_4 + \frac{m_C l_3 l_4}{4} \ddot{q}_3 \cos(q_3 - q_4) - \frac{m_C l_3 l_4}{4} (\dot{q}_3 - \dot{q}_4) \dot{q}_3 \sin(q_3 - q_4) \\ &+ \frac{m_C l_3 l_4}{4} \dot{q}_3 \dot{q}_4 \sin(q_3 - q_4) = -f_{ox} l_4 \sin(q_4) + f_{oy} l_4 \cos(q_4) + T_2, \end{aligned} \tag{4.22}$$

Based on Eq. (4.19), Eq. (4.20), Eq. (4.21), Eq. (4.22) we define the following inertia and non linear damping matrices:

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} \left(\frac{m_A}{3} + m_B\right) l_1^2 & \frac{m_B l_1 l_2}{4}\cos(q_1 - q_2) & 0 & 0 \\ \frac{m_B l_1 l_2}{4}\cos(q_1 - q_2) & \frac{m_B l_2^2}{3} & 0 & 0 \\ 0 & 0 & \frac{m_C l_3^2}{3} & \frac{m_C l_3 l_4}{4}\cos(q_3 - q_4) \\ 0 & 0 & \frac{m_C l_3 l_4}{4}\cos(q_3 - q_4) & \left(\frac{m_D}{3} + m_C\right) l_4^2 \end{bmatrix}$$

where $M(q)$ is the inertia matrix of the pantograph haptic device. Further, the non linear damping matrix is given by

$$\mathbf{D}(\mathbf{q},\dot{\mathbf{q}}) = \begin{bmatrix} -\frac{m_B l_1 l_2}{4}(\dot{q}_1 - \dot{q}_2)\dot{q}_2 \sin(q_1 - q_2) + \frac{m_B l_1 l_2}{4}\dot{q}_1\dot{q}_2 \sin(q_1 - q_2) \\ -\frac{m_B l_1 l_2}{4}(\dot{q}_1 - \dot{q}_2)\dot{q}_1 \sin(q_1 - q_2) + \frac{m_B l_1 l_2}{4}\dot{q}_1\dot{q}_2 \sin(q_1 - q_2) \\ -\frac{m_C l_3 l_4}{4}(\dot{q}_3 - \dot{q}_4)\dot{q}_4 \sin(q_3 - q_4) + \frac{m_C l_3 l_4}{4}\dot{q}_3\dot{q}_4 \sin(q_3 - q_4) \\ -\frac{m_C l_3 l_4}{4}(\dot{q}_3 - \dot{q}_4)\dot{q}_3 \sin(q_3 - q_4) + \frac{m_C l_3 l_4}{4}\dot{q}_3\dot{q}_4 \sin(q_3 - q_4) \end{bmatrix}$$

$$= \underbrace{\begin{bmatrix} \frac{m_B l_1 l_2}{4}\dot{q}_2 s_{12} & -\frac{m_B l_1 l_2}{4}(\dot{q}_1 - \dot{q}_2)s_{12} & 0 & 0 \\ -\frac{m_B l_1 l_2}{4}(\dot{q}_1 - \dot{q}_2)s_{12} & -\frac{m_B l_1 l_2}{4}\dot{q}_1 s_{12} & 0 & 0 \\ 0 & 0 & \frac{m_C l_3 l_4}{4}\dot{q}_4 s_{34} & -\frac{m_C l_3 l_4}{4}(\dot{q}_3 - \dot{q}_4)s_{34} \\ 0 & 0 & -\frac{m_C l_3 l_4}{4}(\dot{q}_3 - \dot{q}_4)s_{34} & \frac{m_C l_3 l_4}{4}\dot{q}_3 s_{34} \end{bmatrix}}_{\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \end{bmatrix}$$

Finally, the control torque input (T) and interaction (T_h) with the operator are defined as follows:

$$\mathbf{T} = \begin{bmatrix} T_1 \\ 0 \\ 0 \\ T_4 \end{bmatrix} \quad \text{and} \quad \mathbf{T_h} = \begin{bmatrix} -f_{ox} l_1 \sin q_1 + f_{oy} l_1 \cos q_1 \\ -f_{ox} l_2 \sin q_2 + f_{oy} l_2 \cos q_2 \\ -f_{ox} l_3 \sin q_3 + f_{oy} l_3 \cos q_3 \\ -f_{ox} l_4 \sin q_4 + f_{oy} l_4 \cos q_4 \end{bmatrix}.$$

Now the equation of motion of the pantograph haptic device can be written in the following compact form:

$$\mathbf{M}(\mathbf{q})\,\ddot{\mathbf{q}} + \mathbf{D}(\mathbf{q},\dot{\mathbf{q}})\,\dot{\mathbf{q}} = \mathbf{T} + \mathbf{T_h} \tag{4.23}$$

It can also be represented in the following form:

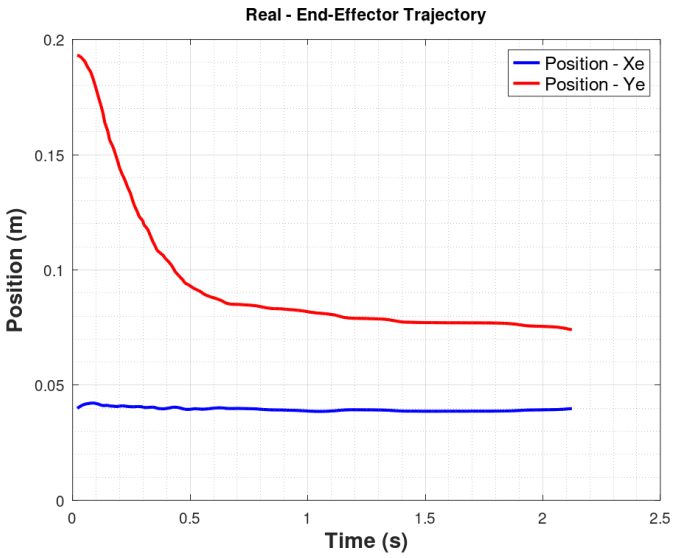
$$\mathbf{M}(\mathbf{q})\,\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\,\dot{\mathbf{q}} = \mathbf{T} + \mathbf{T_h} \tag{4.24}$$

where $\mathbf{M}(\mathbf{q})$ is the inertia matrix ($\mathbf{M}(\mathbf{q})^\top = \mathbf{M}(\mathbf{q}) > 0$). Further, $\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\,\dot{\mathbf{q}}$ accounts for the centrifugal and Coriolis forces.

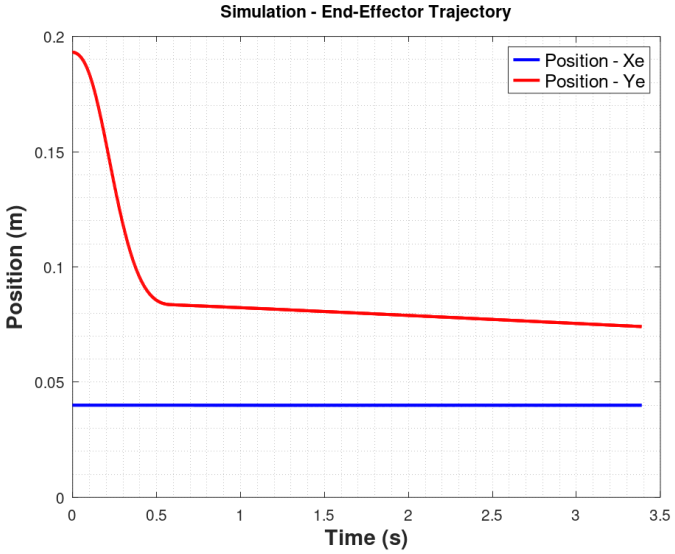
4.2 Validation of the Dynamic equations

To validate the accuracy of the derived dynamic model, a simulation was implemented in Octave, where the complete dynamic equations obtained in Section 4.1 were used to describe the motion of the pantograph mechanism. The simulation replicates the conditions under which the physical system was tested in Chapter 3, where a PID controller was employed for position control. In this setup, the physical robot is replaced by the mathematical model: the same reference trajectories are provided to the PID controller, which computes joint torques. These torques are then applied to the dynamic model, and the system's response specifically, the resulting joint motion is calculated numerically. To assess the validity of the dynamic equations, the simulation results are compared to experimental data recorded from the actual mechanism. More precisely, the comparison focuses on the position of the end-effector and the velocities of the two actuated joints over time. By evaluating the agreement between the simulated and measured trajectories, the accuracy and reliability of the dynamic model in replicating the physical behavior of the system under PID control are thoroughly examined.

The following figure illustrates the position trajectory of the end-effector point as recorded from the physical mechanism under the PID control.



The following figure illustrates the position trajectory of the end-effector point obtained from the simulation, where the system's motion is governed by the dynamic equations and controlled using the same PID controller.

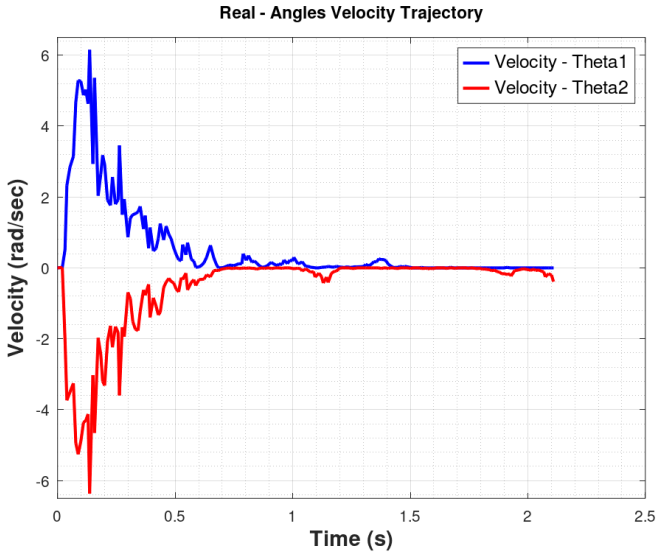


By comparing the two figures, it is observed that the simulated trajectory of the end-effector closely resembles the real one in terms of overall behavior and convergence trend. In both cases, the PID controller effectively drives the end-effector toward a steady-state position.

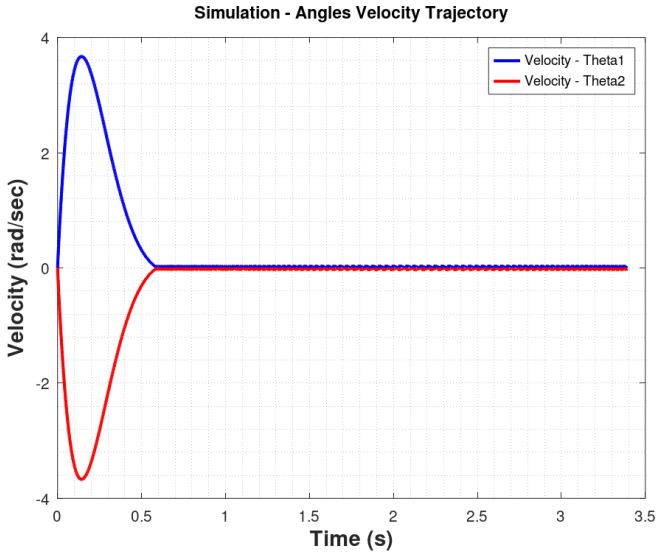
However, minor differences can be noted. In the real system, the trajectory exhibits small oscillations and a slightly slower settling behavior, which can be attributed to unmodeled dynamics, friction, and sensor noise. In contrast, the simulation, based purely on the derived dynamic equations, produces a smoother response with idealized conditions and no external disturbances.

This comparison confirms that the dynamic model captures the essential characteristics of the system's motion and validates its use for simulation and control analysis purposes.

The following figure illustrates the velocity profiles of the two actuated joints as recorded from the physical mechanism under the PID control.



The following figure illustrates the velocity profiles of the two actuated joints obtained from the simulation, where the system's dynamics are governed by the derived dynamic equations and controlled using the same PID controller.



The comparison between the two figures reveals that the simulated joint velocities exhibit a smoother and more idealized response compared to those recorded from the physical mechanism. In the experimental data, the velocities of both joints show notable oscillations and noise, particularly during the initial transient phase, which is expected due to

mechanical imperfections, unmodeled dynamics, and sensor disturbances.

Overall, the qualitative agreement between the trends confirms the validity of the model, although the differences underscore the practical challenges in accurately capturing all physical phenomena in simulation.

Chapter 5

Force Control in a Mass–Spring System

In this chapter, a basic simulation example is presented to illustrate the principles of force control in a compliant environment. Specifically, we consider a one-dimensional mass–spring system, which serves as a simplified model for contact dynamics encountered in robotic manipulation tasks. Unlike position control, where the objective is to track a desired trajectory, force control focuses on regulating the contact force between the system and its environment. The force control example presented here is based on the formulation found in Introduction to Robotics: Mechanics and Control by John J. Craig. This model provides an intuitive and analytically tractable framework for studying the fundamental aspects of force regulation in compliant interaction scenarios.

To this end, we implement a force controller that modifies the motion of the mass so that it applies a desired force on the spring, simulating compliant contact with a surface. The model takes into account the dynamics of the mass and the stiffness of the spring, and incorporates an external disturbance force. This simple scenario allows us to analyze how contact force can be regulated through feedback, and it provides a foundation for understanding more advanced force control schemes used in robotic systems.

5.1 Analysis of the Control System

The following figure illustrates the mass–spring system used for the force control simulation. In this diagram, m represents the mass, k_e is the stiffness of the spring, x denotes the displacement of the mass from its equilibrium position, F is the control force applied to the mass, and F_{dist} is an external disturbance force acting on the system.

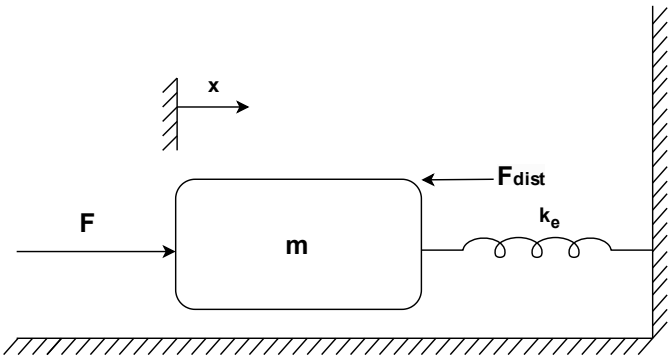


Figure 5.1: Mass-Spring system.

The analysis now focuses on the control of a single mass attached to a spring, as illustrated in Figure 5.1. For completeness, an unknown disturbance force F_{dist} is also considered, which may represent effects such as friction or backlash in gear transmissions. The variable to be controlled namely, the force applied to the environment is, in this model, the force exerted on the spring.

$$f_e = k_e\, x \tag{5.1}$$

The equation that describes this physical system is:

$$f = m\ddot{x} + k_e\, x + f_{\text{dist}} \tag{5.2}$$

By expressing equation Eq. (5.2) as a function of the controlled variable f_e we arrive at the following relation.

$$f = mk_e^{-1}\ddot{f}_e + f_e + f_{\text{dist}} \tag{5.3}$$

Applying the concept of a computed torque controller, we define:

$$a = m\, k_e^{-1} \quad \text{and} \quad \beta = f_e + f_{\text{dist}}$$

Thus, the control law is obtained as follows:

$$f = m\, k_e^{-1}\left[\ddot{f}_d + k_{vf}\dot{e}_f + k_{pf}e_f\right] + f_e + f_{\text{dist}} \tag{5.4}$$

where $e_f = f_d - f_e$ is the force error between the desired force f_d and the actual force f_e If the control law in Eq. (5.4) could be perfectly implemented, the resulting closed-loop equation would be:

$$\ddot{e}_f + k_{vf}\dot{e}_f + k_{pf}e_f = 0 \tag{5.5}$$

However, Eq. (5.4) is not practically feasible, as it requires knowledge of the disturbance force f_{dist}, which is typically unknown. While this term could theoretically be excluded from the control law if a steady-state analysis is considered, this would not generally result in a better or more robust control approach, especially in cases where environmental disturbances are significant, as often encountered in practice.

To simplify the analysis, we choose to neglect the disturbance force by equating Eq. (5.3) and Eq. (5.4). Furthermore, by assuming a steady-state condition—i.e., setting all time derivatives to zero, we arrive at the following relation:

$$e_f = \frac{f_{\text{dist}}}{\alpha} \tag{5.6}$$

Here, $a = m\,k_e^{-1}\,k_{pf}$ represents the active force feedback gain. However, if one chooses to use only the measured force f_e in the control law, instead of the sum $f_e + f_{dist}$ then the steady-state error of the system is given by the following expression:

$$e_f = \frac{f_{\text{dist}}}{1+\alpha} \tag{5.7}$$

In practice, the environment is often compliant, which implies that the value of a tends to be small. As a result, equation Eq. (5.7) can be considered an improved version of Eq. (5.6). Therefore, the control law described in equation Eq. (5.8) is proposed as a more practical solution.

$$f = mk_e^{-1}\left(\ddot{f}_d + k_{vf}\dot{e}_f + k_{pf}e_f\right) + f_d \tag{5.8}$$

However, the primary objective is to regulate the contact force in order to maintain a stable interaction. In practical applications, models that describe contact forces as well-defined functions of time are rare. As a result, the input signals $\ddot{f}_d$ and $\dot{f}_d$ to the control system often become zero or unreliable. Additionally, the measured forces obtained through sensors, tend to be noisy. For this reason, calculating $\dot{f}_e$ numerically through differentiation is generally discouraged.

Although, assuming a linear relationship $f_e = k_e\,x$, it is possible to compute the force derivative based on the velocity $\dot{f}_e = k_e\,\dot{x}$. This is a more realistic assumption, since

velocity measurements are typically more reliable in robotic systems. These two considerations lead to an alternative formulation of the control law, which is expressed as follows:

$$f = m\left(k_{pf}k_e^{-1}e_f - k_{vf}\dot{x}\right) + f_d \tag{5.9}$$

In the block diagram of Figure 5.2, it can be observed that the force estimation errors are used to generate a velocity reference for an inner velocity control loop, with a proportional gain k_{vf}. In some implementations of force control, an additional integral term is included to enhance the system's steady-state performance.

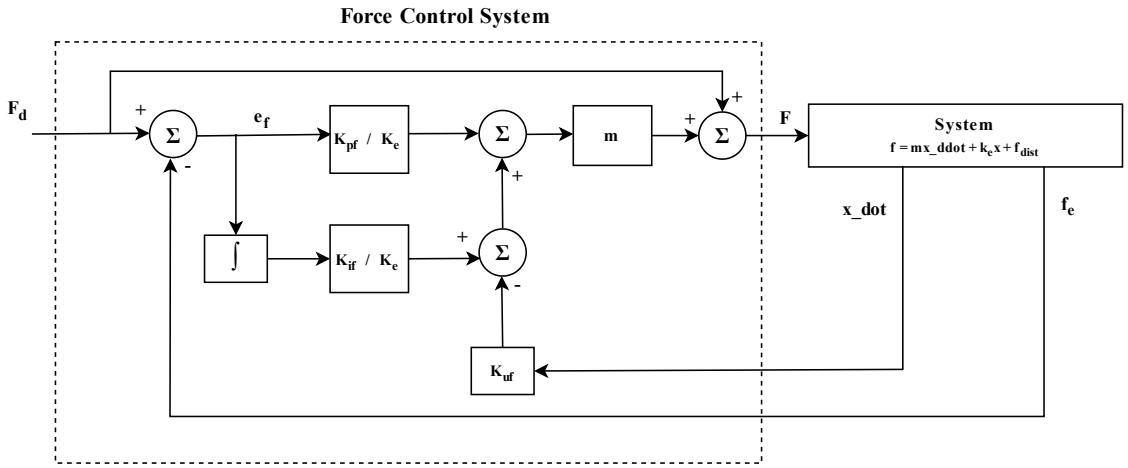


Figure 5.2: Force Control diagram of Mass-Spring system.

5.2 Simulation of the Control System

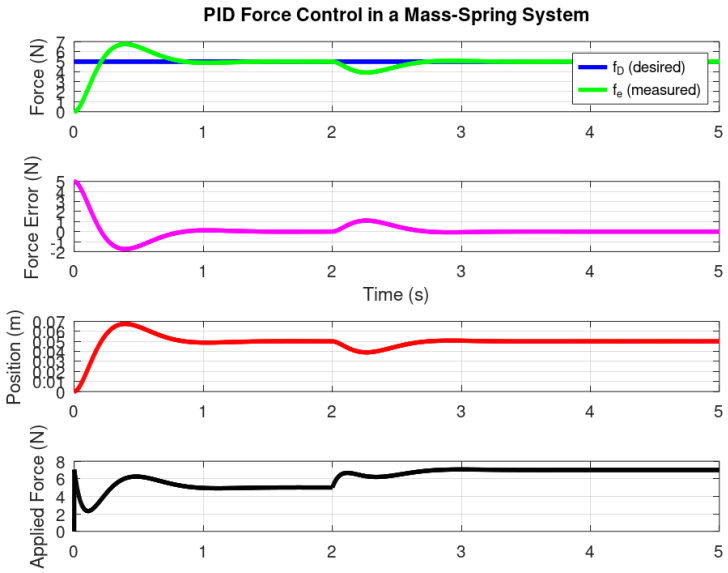
In this section, a simulation is performed to evaluate the behavior of the force control strategy applied to a mass–spring system. The objective is to regulate the contact force between the mass and a compliant environment using a PID-based control scheme. The simulation aims to capture the dynamic response of the system under both reference inputs and external disturbances, and to assess the controller's ability to track the desired force while maintaining system stability and performance. The results provide insight into the effectiveness of the proposed control law in idealized conditions and serve as a foundation for future experimental validation.

The simulation is executed over a total duration of 5 seconds, with a time resolution of 1 millisecond, allowing for a fine-grained analysis of the system's transient and steady-state behavior. The system under control is a one-dimensional mass–spring model, where the

mass is set to $m = 1kg$ and the spring constant is $k_e = 100N/m$, representing a relatively stiff compliant environment.

The control strategy implemented is a PID-based force controller, with proportional, derivative, and integral gains set to $k_{pf} = 40$, $k_{uf} = 20$ and $k_{if} = 5$, respectively. The desired contact force is defined as a constant reference of 5 N throughout the simulation. To evaluate the robustness of the controller, an external disturbance force of 2 N is introduced at $t = 2sec$, simulating an unexpected interaction or environmental variation.

During the simulation, key variables are recorded, including the actual contact force f_e, the force error e_f, the system's position x, and the total control force f applied to the mass.



The figure presents the simulation results of the force control strategy applied to a mass–spring system under the influence of an external disturbance. Each subplot illustrates a key aspect of the system's behavior:

First subplot (Force tracking): This plot compares the desired contact force f_d (blue line) with the actual measured spring force f_e(green line). Initially, the controller successfully drives the system to track the 5 N target, despite a small overshoot. When the 2 N disturbance is introduced at $t = 2sec$, the controller compensates effectively, restoring the desired force with minimal steady-state error.

Second subplot (Force error): The force error e_f initially exhibits a transient re-

sponse with a peak shortly after the simulation begins, then settles close to zero. The disturbance at 2 seconds temporarily increases the error, which is subsequently attenuated by the controller, demonstrating good rejection capabilities.

Third subplot (Position): The position x of the mass reflects the system's compliance. It increases as the force builds up, then stabilizes near the steady-state. At $t = 2sec$, the disturbance causes a small displacement deviation, which is corrected as the controller restores the desired force level.

Fourth subplot (Applied force): This plot shows the control input f applied to the mass. The signal reveals the controller's dynamic response to both the initial transient and the disturbance, with smooth transitions and no abrupt oscillations, indicative of a well-tuned controller.

Chapter 6

Human Hand and Graphical Environment

This chapter focuses on the modeling of the interaction between the haptic device, the human operator, and a virtual graphical environment. One of the main challenges in this context is accurately representing the physical contact between the human hand and the robotic mechanism, which involves complex and often unpredictable dynamics.

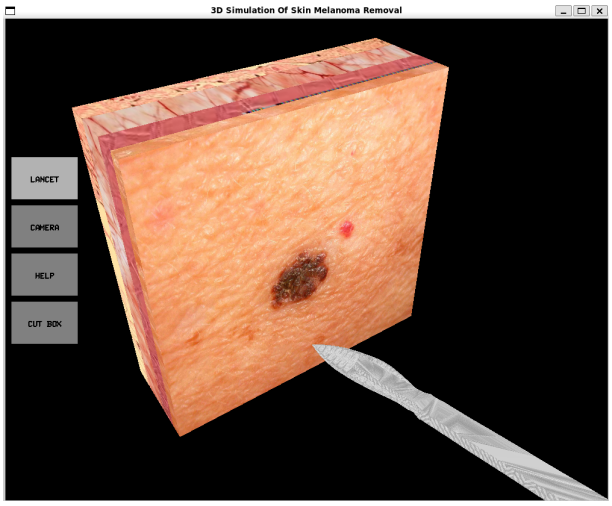


Figure 6.1: Graphical Environment

The desired force that the user should perceive is determined by a virtual simulation environment, which replicates a surgical operation. In this scenario, the user manipulates a scalpel, via the end-effector of the haptic device, to make incisions on simulated human tissue. The control system must ensure that the force experienced by the user realistically reflects the resistance of the virtual skin, as if they were performing the procedure with a real scalpel.

6.1 Modeling of the System

To study and simulate the interaction between the human operator, the haptic device, and the virtual environment, the overall system is modeled as shown in Figure 6.2. The end-effector point of the haptic device is coupled with two distinct subsystems representing the human hand and the virtual environment, respectively.

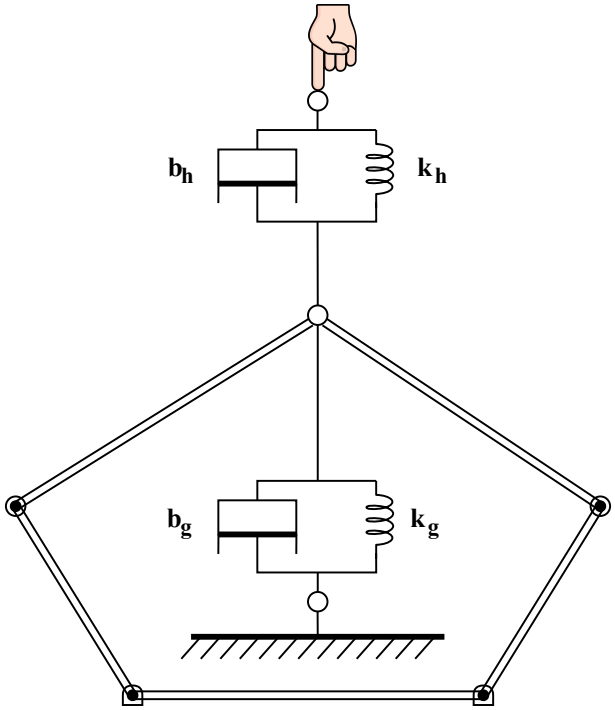


Figure 6.2: System Modeling Diagram

The interaction between the human hand and the end-effector is modeled as a linear spring-damper system characterized by stiffness k_h and damping coefficient b_h. This modeling approach captures the compliant and energy-dissipating nature of human soft tissues and muscles, where the stiffness parameter k_h reflects the hardness of the tissues and muscles during physical contact, while the damping parameter b_h accounts for the energy dissipation caused by internal resistance in the human body during motion. Since the human hand is actively moved by the operator, it acts as a dynamic input to the system.

Similarly, the interaction between the end-effector and the graphical environment is also modeled using a spring-damper system, with parameters k_g and b_g. However, unlike the human hand, the graphical environment is assumed to be stationary. Its role is to generate the desired force that the user should feel. This desired force is being applied to the human hand via the spring-damper coupling.

6.2 System Verification Test

To verify the correct implementation of the mechanical and simulation model, a simple test was conducted. The objective of this test is to examine the system's response to a known constant external force, in the absence of the graphical environment and with the human hand remaining stationary. This setup allows the behavior of the mechanical system to be isolated and observed without external disturbances or active user input.

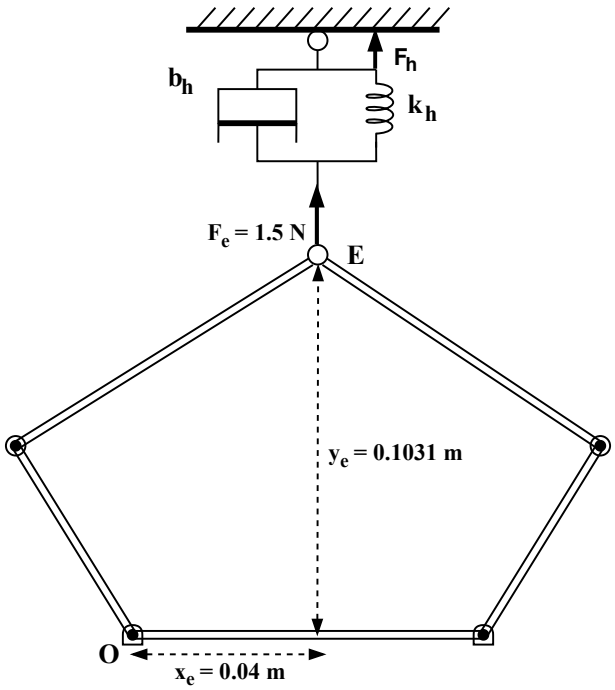
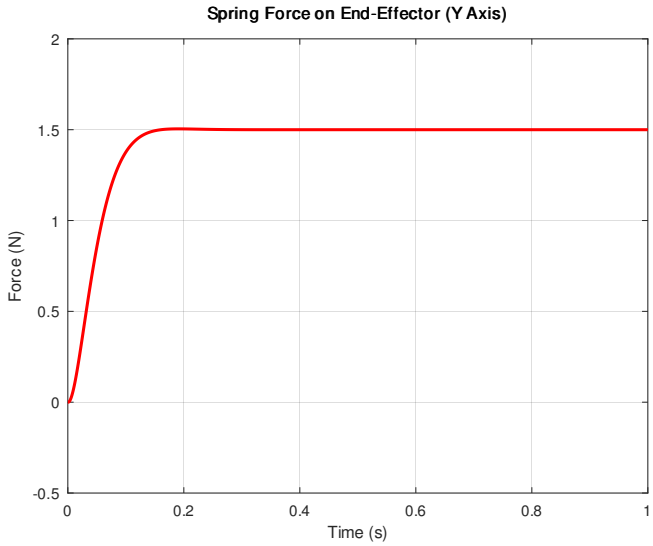
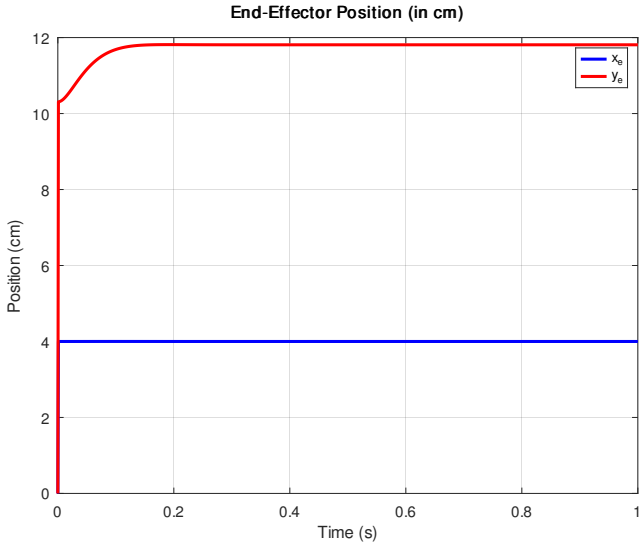


Figure 6.3: Diagram of Test

During the experiment, a constant vertical force (F_e) of 1.5 N was applied to the end-effector of the haptic device. According to the spring-damper modeling approach, the expected outcome is a smooth increase of the spring force (F_h) acting on the end-effector until the system reaches equilibrium. At this point, the spring force should stabilize at 1.5 N, fully counteracting the applied load. As a result, a vertical displacement of the end-effector (point E) is anticipated, corresponding to the deformation of the virtual spring, while the horizontal position remains unchanged due to the direction of the applied force.



The simulation results confirm the expected behavior of the spring force. As shown in the force-time graph, the spring force increases rapidly and smoothly as a response to the constant applied external force. It eventually converges and stabilizes at the value of 1.5 N, indicating that the system reaches a steady-state equilibrium. This confirms that the spring-damper model accurately captures the force dynamics in the absence of disturbances.



In parallel, the position-time graph reveals the expected response of the end-effector's position. Specifically, the vertical component of the end-effector's position increases smoothly due to the spring deformation and stabilizes at a new constant value, while the horizontal position remains unchanged. The end-effector is observed to move vertically by approximately 2 cm. This behavior validates the correctness of the mechanism's kinematic response and confirms the accuracy of the implemented model in simulating the end-effector displacement under static loading.

Chapter 7

Force Control in the Haptic Device

This chapter presents the implementation of force control in the haptic device using a PID controller. The objective is to enhance the user's force perception through accurate and stable interaction. A key distinction between simulation and hardware implementation is the absence of a force sensor in the physical device, which requires alternative strategies for estimating and compensating interaction forces. The following sections describe the control architecture, simulation results, and methods for improving force realism both in simulation and on the real mechanism.

7.1 PID Force Controller

The force-based PID controller used in this study follows the same structure as the position-based PID controller introduced in Chapter 3. The main differences lie in the nature of the input and the computation of the control error. In this case, the input is the desired force F_D, which is generated by the graphical environment. The control error e_f is calculated as the difference between the desired force and the actual force felt by the human hand, which corresponds to the force generated by the upper spring of stiffness k_h.

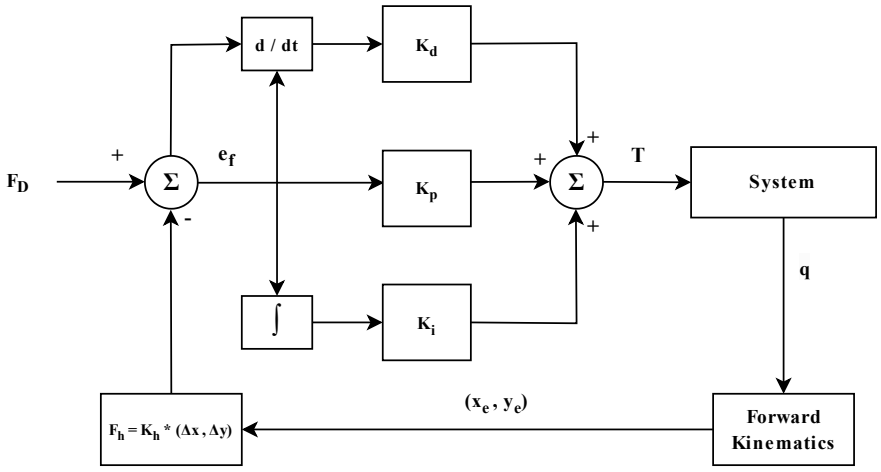


Figure 7.1: PID Force Controller Diagram

To properly simulate the effect of force loss in the virtual environment and enable realistic force control, a portion of the total torque delivered to the actuators is intentionally reduced. Specifically, 40% of the commanded torque is subtracted before it is applied to the motors. This artificial loss allows the PID controller to actively compensate and drive the end-effector toward the correct force feedback.

7.2 Description of Simulation Procedure and Results

In the implemented simulation, the initial position of the hand coincides with that of the end-effector. An initial downward velocity of 0.01 m/s is applied to the hand. Based on the graphical interface, the hand is expected to feel a positive force in the upward direction. The force controller ensures that the end-effector applies the appropriate force so that the human hand perceives the desired spring force F_h. This procedure is illustrated in the figure below.

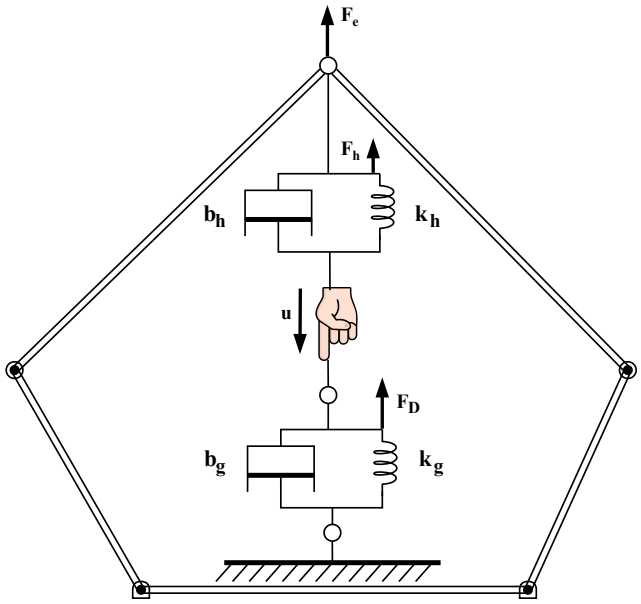
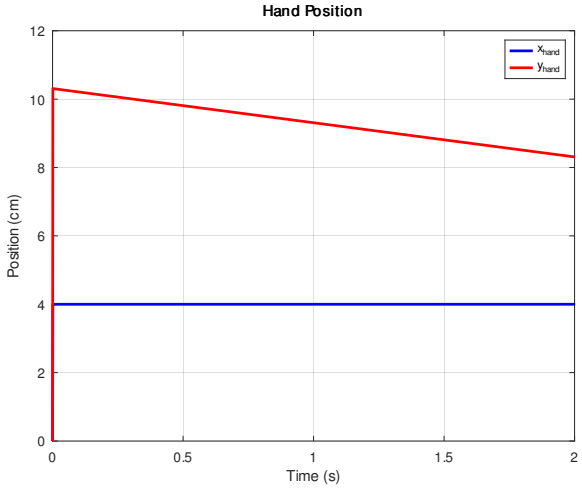
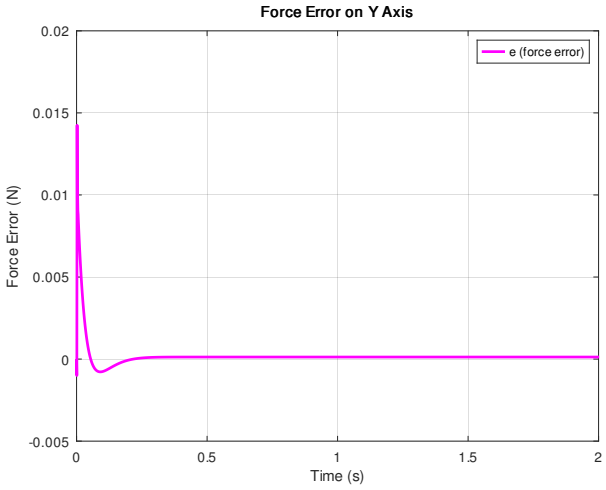


Figure 7.2: Simulation Diagram

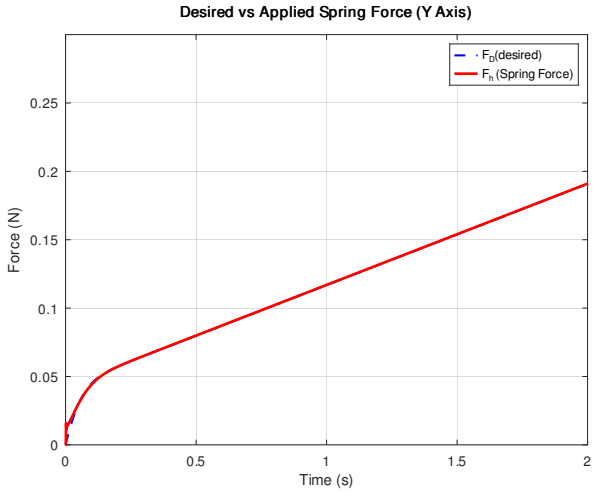
Subsequently, the graphical plots of the simulation results are presented.

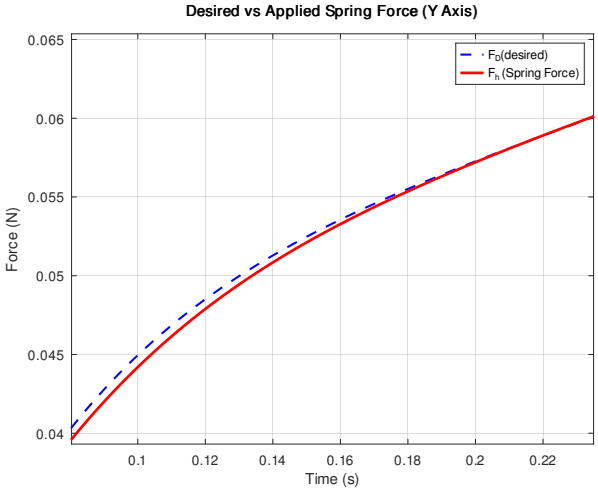


The first graph illustrates the position of the human hand over time. As expected, the vertical component of the hand's position decreases smoothly along the y-axis, confirming the initial downward movement applied to the hand, while the horizontal position remains constant. This behavior verifies the correct implementation of the simulated hand motion.



The second graph shows the force error over time. The error decreases smoothly and rapidly approaches zero, indicating that the PID force controller successfully minimizes the difference between the desired and actual forces, ensuring accurate force tracking during the simulation.



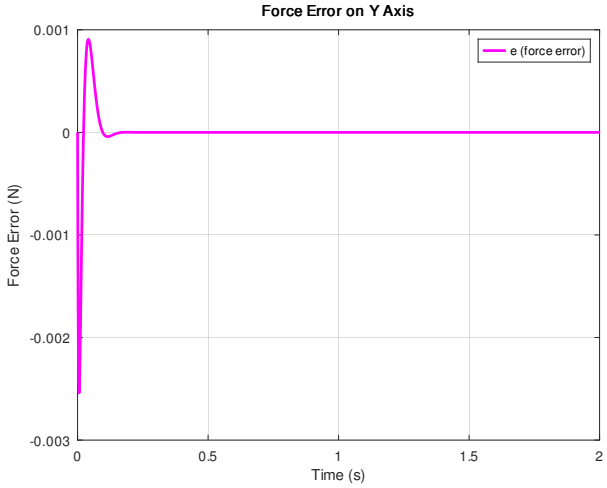


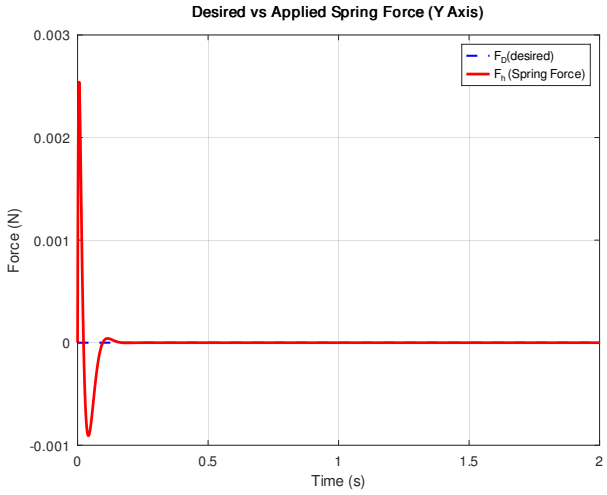
The previously presented force error was computed as the difference between the desired spring force and the actual force applied by the spring. In the figure above, it is observed that the applied spring force increases smoothly and rapidly, closely following the desired force throughout the simulation. This behavior confirms the successful operation of the force controller. The second graph is a zoomed-in version of the first, providing a clearer view of the transient response and the convergence between the two force signals.

7.3 Realistic Enhancement of Force Perception in Simulation

In the absence of a force sensor in the system, the perceived force must be reconstructed based solely on the system's model. This constraint enables the implementation of a simplified and effective method for improving force perception by compensating for static friction.

Since static friction acts as a predictable and easily measurable disturbance, it can be cancelled virtually in simulation. The approach assumes that no desired force is being sent by the graphical environment (or that it is zero), and that the user simply pushes the end-effector. Under these conditions, eliminating static friction ensures that there is no artificial resistance to motion.

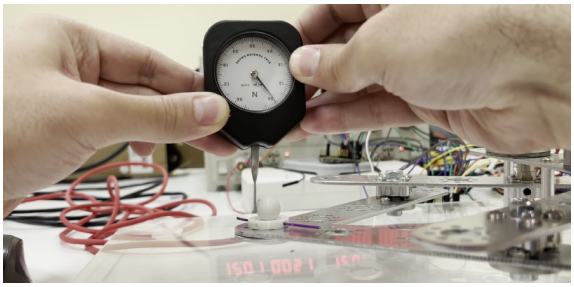




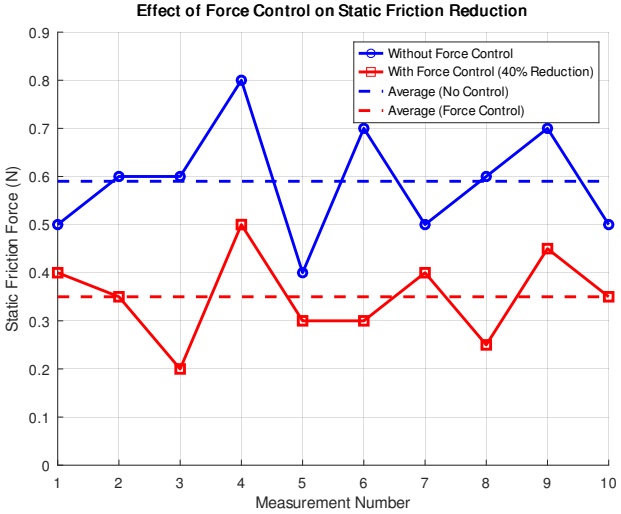
The results demonstrate the effectiveness of the static friction cancellation. In this case, the desired force is zero, meaning the user should not feel any resistance. As shown in the force error graph, the error rapidly converges to zero, indicating precise force tracking. Similarly, the applied spring force stabilizes around zero, confirming that the user perceives no artificial resistance during the motion.

7.4 Static Friction Compensation on the Real Mechanism

In the real mechanism, there is no physical spring or damper between the human hand and the end-effector, unlike the simulated model. As a result, a different approach is required to enhance force perception. The influence of static friction was measured using a dynamometer by simply pushing the end-effector without user involvement. The average force required to overcome static friction was approximately 0.6 N.



To reduce this resistance and improve the responsiveness of the system, the end-effector was programmed to apply an assisting force in the same direction as the user's input, effectively canceling out a portion (40%) of the static friction. In this configuration, the end-effector slightly "pulls" the user's hand, helping it initiate motion more easily. It should be noted that eliminating 100% of the static friction would result in the end-effector moving on its own without any input from the hand, an undesirable outcome for realistic interaction.



The diagram illustrates the measured static friction forces across ten trials, both with and without the application of force control. The blue line represents the unassisted case, where the average force required to overcome static friction is around 0.6 N. In contrast, the red line shows the effect of applying force control that compensates for 40% of the friction. This results in a clear reduction of the required force, with the average dropping to approximately 0.36 N. The comparison confirms the effectiveness of the control strategy in reducing the user's effort to initiate motion.

Bibliography

[1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th Greek ed., Tziola Publications, Thessaloniki, 2020.

[2] K. Ogata, *Modern Control Engineering*, 5th ed., Prentice Hall, Upper Saddle River, NJ, 2010.

[3] B. J. Gamble, "5-Bar Linkage Kinematic Solver and Simulator," Honors College Senior Thesis, University of Vermont, Burlington, VT, USA, 2020. [Online]. Available: https://scholarworks.uvm.edu/hcoltheses/406

[4] I. Chavdarov, "Kinematics and Force Analysis of a Five-Link Mechanism by the Four Spaces Jacoby Matrix," *Problems of Engineering Cybernetics and Robotics*, vol. 55, Bulgarian Academy of Sciences, Sofia, 2005. [Online]. Available: https://www.researchgate.net/publication/290393600

[5] M. Abdel Ghany, M. Sallam, M. Shamseldin, and L. El-Tehewy, "Mathematical Model of Two-DOF Five Links Closed-Chain Mechanism (Pantograph)," *International Journal of Advanced Engineering and Business Sciences*, vol. 2, no. 1, pp. 60–74, Apr. 2021. doi: 10.21608/ijaebs.2021.75247.1013

[6] I. S. M. Khalil and M. Abu Seif, "Modeling of a Pantograph Haptic Device," Technical Report, University of Twente, Enschede, The Netherlands, 2015. [Online]. Available: https://www.researchgate.net/publication/308996435_Modeling_of_a_Pantograph_Haptic_Device

Appendix A

Symbolic Solution of Differential Kinematics (Jacobian Matrix) using Python

Below is the Python code that represents and calculates the symbolic solution of differential kinematics. Specifically, it calculates the velocities along the x and y axes of the end-effector point.

By examining the code, one can observe that the solutions of Forward Kinematics (Section 2.1) and the representation of angles θ_3 and θ_4 in terms of the motor angles (Section 2.4) are computed first, as these are used in the calculation of the velocities.

```
1
2   import sympy as sym
3   from sympy import diff, sin, cos, sqrt, acos, pi
4
5   theta1 = sym.Symbol('theta1')
6   theta2 = sym.Symbol('theta2')
7   theta1_dot = sym.Symbol('theta1_dot')
8   theta2_dot = sym.Symbol('theta2_dot')
9
10  l0 = sym.Symbol('l0')
11  l1 = sym.Symbol('l1')
12  l2 = sym.Symbol('l2')
13  l3 = sym.Symbol('l3')
14  l4 = sym.Symbol('l4')
15
16  a = ( ( sin(theta1)*l1 - sin(theta2)*l4 ) / ( l0 + \
17  cos(theta2)*l4  - cos(theta1)*l1 ) )
18
19  b = ( ( (sin(theta2))**2 * l4**2 + (l0 + cos(theta2)*l4)**2 \
20  - l1**2 + l2**2 - l3**2 ) / ( 2*(l0 + cos(theta2)*l4 \
```

```
21  - cos(theta1)*l1) ) )
22
23  c = ( (a**2) + 1 )
24  d = ( (2*a*b) - (2*a*cos(theta1)*l1) - (2*sin(theta1)*l1) )
25  e = ( b**2 - 2*b*cos(theta1)*l1 + l1**2 - l2**2 )
26
27  ye = ( ( -d + sqrt(d**2 - 4*c*e) ) / ( 2*c ) )
28  xe = ( (a*ye) + b )
29
30  theta3 = pi - acos( (-(xe)**2 - (ye)**2 + 2*xe*l0 \
31  - (l0)**2 + (l3)**2 + (l4)**2 ) / (2*l3*l4) )
32
33  theta4 = pi + acos( (-(xe)**2 - (ye)**2 + (l1)**2 \
34  + (l2)**2 ) / (2*l1*l2) )
35
36  a00 = l1*sin(theta1) + l2*(sin(theta1)*cos(theta4) \
37  + cos(theta1)*sin(theta4))
38
39  a01 = l2*(sin(theta1)*cos(theta4) + cos(theta1) \
40  *sin(theta4))
41
42  a10 = l1*cos(theta1) + l2*(cos(theta1)*cos(theta4) \
43  - sin(theta1)*sin(theta4))
44
45  a11 = l2*(cos(theta1)*cos(theta4) - sin(theta1) \
46  *sin(theta4))
47
48  b00 = l4*sin(theta2) + l3*(sin(theta2)*cos(theta3) \
49  + cos(theta2)*sin(theta3))
50
51  b01 = l3*(sin(theta2)*cos(theta3) + cos(theta2) \
52  *sin(theta3))
53
54  b10 = l4*cos(theta2) + l3*(cos(theta2)*cos(theta3) \
55  - sin(theta2)*sin(theta3))
56
57  b11 = l3*(cos(theta2)*cos(theta3) - sin(theta2) \
58  *sin(theta3))
59
60  xe_dot = -( b01*( (a10*a01 - a11*a00) / (b11*a01 \
61  - a11*b01) ) )*theta1_dot + ( a01*( \
62  (b10*b01 - b11*b00) / (b11*a01 - \
63  a11*b01) ) )*theta2_dot
64
65  ye_dot = ( ( (a10*a01 - a11*a00) / (a01) ) + ( \
66  (a11*b01*(a10*a01 - a11*a00)) / (a01* \
67  (b11*a01 - a11*b01)) ) )*theta1_dot - \
68  ( a11*( (b10*b01 - b11*b00) / (b11*a01 \
69  - a11*b01) ) )*theta2_dot
70
```

Appendix B

Numerical Solution of Differential Kinematics (Jacobian Matrix) using Python

As previously mentioned, the velocities along the two axes of the end-effector point can be easily found by differentiating the position equations obtained through Forward Kinematics (Section 2.1).

Below is a simple Python program that uses *diff* function provided by *sympy* library in Python. Variables f00, f01, f10 and f11 constitute the four elements of Jacobian matrix.

```
1
2   import sympy as sym
3   from sympy import diff, sin, cos, sqrt
4
5   theta1 = sym.Symbol('theta1')
6   theta2 = sym.Symbol('theta2')
7   theta1_dot = sym.Symbol('theta1_dot')
8   theta2_dot = sym.Symbol('theta2_dot')
9
10  l0 = sym.Symbol('l0')
11  l1 = sym.Symbol('l1')
12  l2 = sym.Symbol('l2')
13  l3 = sym.Symbol('l3')
14  l4 = sym.Symbol('l4')
15
16  a = ( ( sin(theta1)*l1 - sin(theta2)*l4 ) / ( l0 + \
17  cos(theta2)*l4 - cos(theta1)*l1 ) )
18
19  b = ( ( (sin(theta2))**2 * l4**2 + (l0 + \
20  cos(theta2)*l4)**2 - l1**2 + l2**2 - l3**2 ) / \
```

```
21  ( 2*(l0 + cos(theta2)*l4 - cos(theta1)*l1) ) )
22
23  c = ( (a**2) + 1 )
24  d = ( (2*a*b) - (2*a*cos(theta1)*l1) - (2*sin(theta1)*l1) )
25  e = ( b**2 - 2*b*cos(theta1)*l1 + l1**2 - l2**2 )
26
27  ye = ( ( -d + sqrt(d**2 - 4*c*e) ) / ( 2*c ) )
28  xe = ( (a*ye) + b )
29
30  f00 = diff(xe,theta1)
31  f10 = diff(ye,theta1)
32  f01 = diff(xe,theta2)
33  f11 = diff(ye,theta2)
34
35  xe_dot = f00*theta1_dot + f01*theta2_dot
36  ye_dot = f10*theta1_dot + f11*theta2_dot
37
```

Appendix C

PID Position Controller using Python

Below is the Python code implementing the PID position controller analyzed in Chapter 3.

```
1
2   import nidaqmx
3   from nidaqmx.constants import LineGrouping
4   from nidaqmx.constants import EncoderType, AngleUnits
5   import time
6   from configure_encoder import configure_encoder
7   import math
8   from inverse_kinematics import inverse_kinematics
9   from forward_kinematics import forward_kinematics
10
11  # Device and channel configuration
12  device_name_1 = "Dev1"
13  analog_channel_1 = f"{device_name_1}/ao1"  # Analog Output channel AO1
14  analog_channel_2 = f"{device_name_1}/ao0"  # Analog Output channel AO0
15  digital_channel_name = f"{device_name_1}/port0/line0"  # Digital Output
                                    channel (P0.0)
16
17  device_name_2 = "Dev2"                          # Device name for the
                                    encoders
18  channel_name0 = f"{device_name_2}/ctr0"        # Counter 0 for the first
                                    encoder
19  channel_name1 = f"{device_name_2}/ctr1"        # Counter 1 for the second
                                    encoder
20  initial_angle = 90.0
21  pulses_per_rev = 9750
22  a0_input_term = "PFI0"
23  b0_input_term = "PFI1"
24  a1_input_term = "PFI2"
25  b1_input_term = "PFI3"
```

```
26
27  reading_task_0 = nidaqmx.Task()
28  reading_task_1 = nidaqmx.Task()
29  analog_task_a01 = nidaqmx.Task()
30  analog_task_a00 = nidaqmx.Task()
31  digital_task = nidaqmx.Task()
32
33  configure_encoder(reading_task_0, channel_name0, a0_input_term,
                                      b0_input_term, pulses_per_rev,
                                      initial_angle)
34  configure_encoder(reading_task_1, channel_name1, a1_input_term,
                                      b1_input_term, pulses_per_rev,
                                      initial_angle)
35
36  sampling_period = 1 / 200   # Define the sampling period: 200 Hz (5 ms per
                                  sample)
37
38  # Voltages to output
39  output_voltage_a00 = 0
40  output_voltage_a01 = 0
41  digital_enable = True
42
43  x_d = 0.0400
44  y_d = 0.0731
45  theta1 = theta2 = math.radians(initial_angle)
46  theta1_d, theta2_d = inverse_kinematics(x_d, y_d)
47
48  error1 = theta1_d - theta1
49  error2 = theta2_d - theta2
50
51  # PID controller gains
52  Kp = 0.06
53  Kd = 0.01
54  Ki = 0.04
55
56  # Initialize previous errors, integrals, and time for PID control
57  prev_error1 = error1
58  prev_error2 = error2
59  integral_error1 = 0
60  integral_error2 = 0
61  prev_time = time.time()
62
63  prev_theta1 = theta1
64  prev_theta2 = theta2
65
66  try:
67
68  reading_task_0.start()
69  reading_task_1.start()
70
71  # Add an analog output voltage channel for AO1
72  analog_task_a01.ao_channels.add_ao_voltage_chan(
73  physical_channel=analog_channel_1,
```

```
74  min_val=-9.5,
75  max_val=9.5
76  )
77
78  # Add an analog output voltage channel for AO0
79  analog_task_a00.ao_channels.add_ao_voltage_chan(
80  physical_channel=analog_channel_2,
81  min_val=-9.5,
82  max_val=9.5
83  )
84
85  # Add a digital output channel
86  digital_task.do_channels.add_do_chan(
87  lines=digital_channel_name,
88  line_grouping=LineGrouping.CHAN_PER_LINE
89  )
90
91  digital_task.write(digital_enable, auto_start=True)
92
93  xe, ye = forward_kinematics(theta1, theta2)
94
95  output_volt_file = open("output_volt.txt", "w")
96  output_volt_file.write("V0, V1\n")  # Add header to the file
97
98  error_file = open("error.txt", "w")
99  error_file.write("error1, error2, time\n")  # Add header to the file
100
101 velocity_file = open("velocity.txt", "w")
102 velocity_file.write("velocity1, velocity2, time\n") # Add header to the
                                      file
103
104 position_error_file = open("position_error.txt", "w")
105 position_error_file.write("x_error, y_error, time\n") # Add header to the
                                      file
106
107 position_file = open("position.txt", "w")
108 position_file.write("xe, ye, time\n") # Add header to the file
109
110 position_deg_file = open("position_deg.txt", "w")
111 position_deg_file.write("theta1, theta2, time\n") # Add header to the file
112
113 error_time_start = time.time()
114
115 while time.time() - error_time_start < 20:
116
117 start_time = time.time()
118
119 # Calculate time elapsed since last loop
120 delta_time = start_time - prev_time
121
122 velocity1 = (theta1 - prev_theta1) / delta_time if delta_time > 0 else 0
123 velocity2 = (theta2 - prev_theta2) / delta_time if delta_time > 0 else 0
124 velocity_file.write(f"{velocity1:.5f}, {velocity2:.5f}, {time.time() -
```

```
                                        error_time_start:.5f}\n")
125
126     # Integral term for errors
127     integral_error1 += error1 * delta_time
128     integral_error2 += error2 * delta_time
129
130     # Derivative term for errors
131     if delta_time > 0:
132     d_error1 = (error1 - prev_error1) / delta_time
133     d_error2 = (error2 - prev_error2) / delta_time
134     else:
135     d_error1 = 0
136     d_error2 = 0
137
138     # PID Control calculations
139     T1 = Kp * error1 + Kd * d_error1 + Ki * integral_error1
140     T2 = Kp * error2 + Kd * d_error2 + Ki * integral_error2
141
142     # Convert torques to output voltages
143     output_voltage_a00 = 91.35 * T1
144     output_voltage_a01 = 91.35 * T2
145
146     # Voltage saturation [-10V, 10V]
147     output_voltage_a00 = max(min(output_voltage_a00, 9.5), -9.5)
148     output_voltage_a01 = max(min(output_voltage_a01, 9.5), -9.5)
149
150     # Write voltages to file
151     output_volt_file.write(f"{output_voltage_a00:.4f}, {output_voltage_a01:.4f}
                                        \n")
152
153     # Send voltages to the analog outputs
154     analog_task_a00.write(output_voltage_a00, auto_start=True)
155     analog_task_a01.write(output_voltage_a01, auto_start=True)
156
157     prev_theta1 = theta1
158     prev_theta2 = theta2
159
160     # Read angular positions from encoders
161     theta1 = math.radians(reading_task_0.read())
162     theta2 = math.radians(reading_task_1.read())
163     #print(f"Theta1: {theta1:.5f} rad, Theta2: {theta2:.5f} rad")
164     theta1_deg = math.degrees(theta1)
165     theta2_deg = math.degrees(theta2)
166     position_deg_file.write(f"{theta1_deg:.5f}, {theta2_deg:.5f}, {time.time()
                                        - error_time_start:.5f}\n")
167
168     prev_error1 = error1
169     prev_error2 = error2
170
171     # Update errors
172     error1 = theta1_d - theta1
173     error2 = theta2_d - theta2
174     error1_deg = math.degrees(error1)
```

```
175     error2_deg = math.degrees(error2)
176     print(f"error_theta1: {abs(error1_deg):.5f} deg, error_theta2: {abs(
                                        error2_deg):.5f} deg\n")
177     error_time = time.time() - error_time_start
178     error_file.write(f"{abs(error1_deg):.5f}, {abs(error2_deg):.4f}, {
                                        error_time:.5f}\n")
179
180     # Update position for logging
181     xe, ye = forward_kinematics(theta1, theta2)
182     position_file.write(f"{xe:.5f}, {ye:.5f}, {time.time() - error_time_start:.
                                        5f}\n")
183     print(f"X: {xe:.4f} m, Y: {ye:.4f} m")
184     error_x = x_d - xe
185     error_y = y_d - ye
186     print(f"error_x: {abs(error_x)*100:.5f} cm, error_y: {abs(error_y)*100:.5f}
                                         cm\n")
187     position_error_file.write(f"{abs(error_x):.5f}, {abs(error_y):.5f}, {time.
                                        time() - error_time_start:.5f}\n")
188
189     # Update time for next iteration
190     prev_time = start_time
191
192     # Maintain constant loop frequency
193     elapsed_time = time.time() - start_time
194     sleep_time = sampling_period - elapsed_time
195     if sleep_time > 0:
196     time.sleep(sleep_time)
197
198     print("---------------------------------")
199
200     output_voltage_a00 = 0
201     output_voltage_a01 = 0
202     analog_task_a01.write(output_voltage_a01, auto_start=True)
203     analog_task_a00.write(output_voltage_a00, auto_start=True)
204     digital_enable = False
205     digital_task.write(digital_enable, auto_start=True)
206
207     except KeyboardInterrupt:
208     output_voltage_a00 = 0
209     output_voltage_a01 = 0
210     analog_task_a01.write(output_voltage_a01, auto_start=True)
211     analog_task_a00.write(output_voltage_a00, auto_start=True)
212     digital_enable = False
213     digital_task.write(digital_enable, auto_start=True)
214     print("Stopped.")  # Handle program termination with Ctrl+C
215
216     finally:
217     # Stop and close the tasks
218     analog_task_a01.stop()
219     analog_task_a01.close()
220     analog_task_a00.stop()
221     analog_task_a00.close()
222     digital_task.stop()
```

```
223     digital_task.close()
224     reading_task_0.stop()
225     reading_task_0.close()
226     reading_task_1.stop()
227     reading_task_1.close()
228     print("Tasks stopped and closed.")
229
230
```

Appendix D

PID Force Controller using Octave

Below is the Octave code implementing the PID force controller analyzed in Chapter 7.

Code Listing D.1: Octave script

```
%% --- Force Control Simulation with Moving Hand and Dynamic Desired Force ---

clear; clc; close all;

% Simulation time
T = 5;
dt = 0.001;
t = 0:dt:T;

% Spring and damping parameters
k_e = [100; 100];  % Hand spring constants
b_e = [10; 10];    % Hand damping constants
k_des = [8; 8];    % Desired force spring
b_des = [5; 5];    % Desired force damping

% Geometry
l1 = 0.08; l2 = 0.12; l3 = 0.12; l4 = 0.08; l0 = 0.08;

% Initial joint conditions
q = zeros(4, length(t));
dq = zeros(4, length(t));

q(:,1) = [2.53011829042451; 0.620103; 2.521490; 0.611474363165284];
```

```
24          dq(:,1) = [0; 0; 0; 0];
25
26          % Hand initial state
27          x_ref = 0.04;
28          y_ref = 0.1031;
29          x_hand = x_ref;
30          y_hand = y_ref;
31          v_hand = [0; -0.01];
32
33          % Storage arrays
34          xe_arr = zeros(1, length(t));
35          ye_arr = zeros(1, length(t));
36          v_e_arr = zeros(2, length(t));
37          f_e_arr = zeros(2, length(t));
38          F_control_arr = zeros(2, length(t));
39          F_applied_arr = zeros(2, length(t));
40          tau1_arr = zeros(1, length(t));
41          tau4_arr = zeros(1, length(t));
42
43          int_e_force = zeros(2, length(t));
44          d_e_force = zeros(2, length(t));
45          I_term_arr = zeros(2, length(t));
46
47          x_hand_arr = zeros(1, length(t));
48          y_hand_arr = zeros(1, length(t));
49          delta_hand_arr = zeros(2, length(t));
50          delta_desired_arr = zeros(2, length(t));
51          f_h_arr = zeros(2, length(t));
52
53          for k = 2:length(t)
54                  q1 = q(1,k-1); q2 = q(2,k-1); q3 = q(3,k-1); q4 = q
                        (4,k-1);
55
56                  % Update hand position
57                  x_hand = x_hand + v_hand(1)*dt;
58                  y_hand = y_hand + v_hand(2)*dt;
59
60                  x_hand_arr(k) = x_hand;
61                  y_hand_arr(k) = y_hand;
62
63                  % Inertia matrix
64                  M = [
65                  0.000853, 0.00024*cos(q1 - q2), 0, 0;
66                  0.00024*cos(q1 - q2), 0.00048, 0, 0;
67                  0, 0, 0.00048, 0.00024*cos(q3 - q4);
68                  0, 0, 0.00024*cos(q3 - q4), 0.000853];
69
70                  % Coriolis matrix
71                  dq1 = dq(1,k-1); dq2 = dq(2,k-1); dq3 = dq(3,k-1);
```

```
                dq4 = dq(4,k-1);
72              C = [
73              0.00024*dq2*sin(q1 - q2), (-0.00024*dq1 + 0.00024*dq2
                    )*sin(q1 - q2), 0, 0;
74              (-0.00024*dq1 + 0.00024*dq2)*sin(q1 - q2), -0.00024*
                    dq1*sin(q1 - q2), 0, 0;
75              0, 0, -0.00024*dq4*sin(q3 - q4), (0.00024*dq4 -
                    0.00024*dq3)*sin(q3 - q4);
76              0, 0, (0.00024*dq4 - 0.00024*dq3)*sin(q3 - q4),
                    0.00024*dq3*sin(q3 - q4)];
77
78              % Kinematics
79              [xe, ye, jacobian] = fivebar_kinematics(q1, q4, l0,
                    l1, l2, l3, l4);
80              xe_arr(:,k) = xe;
81              ye_arr(:,k) = ye;
82
83              v_e_now = jacobian * [dq(1,k-1); dq(4,k-1)];
84              v_e_arr(:,k) = v_e_now;
85
86              % Relative extension and velocity
87              delta_x = xe - x_hand;
88              delta_y = ye - y_hand;
89              delta_hand_arr(:,k) = [delta_x; delta_y];
90              delta_vx = v_e_now(1) - v_hand(1);
91              delta_vy = v_e_now(2) - v_hand(2);
92
93              % Force on end-effector from spring
94              f_e = k_e .* [delta_x; delta_y];
95              f_e_arr(:,k) = f_e;
96
97              % Desired force calculation
98              F_control_desired = - k_des .* ([xe; ye] - [x_ref;
                    y_ref]) - b_des .* v_e_now;
99
100             delta_desired_arr(:,k) = [xe - x_ref; ye - y_ref];
101             F_control_arr(:,k) = F_control_desired;
102
103             % PID Force Control
104             error_force = F_control_desired - f_e;
105             int_e_force(:,k) = int_e_force(:,k-1) + error_force *
                     dt;
106             d_e_force(:,k) = (error_force - (F_control_desired -
                    f_e_arr(:,k-1))) / dt;
107
108             Kp_force = diag([25; 25]);
109             Ki_force = diag([1000; 1000]);
110             Kd_force = diag([0.2; 0.2]);
111             F_control_applied = Kp_force * error_force + Ki_force
```

```
                    * int_e_force(:,k) + Kd_force * d_e_force(:,k);
112
113             I_term = Ki_force * int_e_force(:,k);
114             I_term_arr(:,k) = I_term;
115
116             % Disturbance
117             alpha = 0.6;
118
119             F_applied_arr(:,k) = F_control_applied;
120
121             % Torque calculation
122             tau = jacobian' * F_control_applied;
123             tau_full = [tau(1); 0; 0; tau(2)];
124             tau_full = alpha * tau_full;
125             tau1_arr(k) = tau_full(1);
126             tau4_arr(k) = tau_full(4);
127
128             f_h = -k_e .* [delta_x; delta_y] - b_e .* [delta_vx;
                    delta_vy];
129             f_h_arr(:,k) = f_h;
130             tau_h = jacobian' * f_h;
131             T_h = [tau_h(1); 0; 0; tau_h(2)];
132
133             q_ddot = M \ (tau_full + T_h - C * dq(:,k-1));
134
135             % State integration
136             dq(:,k) = dq(:,k-1) + q_ddot * dt;
137             q(:,k) = q(:,k-1) + dq(:,k) * dt;
138         end
```